



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

GENERATIVE AI

LABORATORY PRACTICAL RECORD

UNIT IV — Multimodal Generative AI Applications

10 Practical Implementation Reports — Chatbots, Text-to-Image, Speech-to-Text, Text-to-Speech, Summarization, Translation, Resume Screening & Research Assistance

Submitted by:

C.PRAVEEN KUMAR

192472034

CSE-AI

Course Code: CSA6502

Course Name: Generative AI

Academic Year: [2024–2028]

QUESTION 1: AI CHATBOT FOR ENGINEERING COLLEGE STUDENT QUERIES

Develop an AI chatbot that answers student queries related to an engineering college using a pre-trained language model.

Folder Name	Q1_College_Chatbot
Python File	chatbot.py
AI Service / Method	Hugging Face Transformers — google/flan-t5-small

1. Aim

To develop an AI chatbot that accepts engineering-college-related student queries and generates concise responses using a pre-trained language model.

2. Objective

- To provide a simple conversational interface for students.
- To use a pre-trained language model without training a model from scratch.
- To accept natural-language questions related to college courses, departments, examinations, library, laboratories, and campus facilities.
- To display the generated response through a Streamlit web application.

3. Software and Hardware Requirements

Software Requirements

- Python 3.x
- Visual Studio Code
- Streamlit
- Hugging Face Transformers
- PyTorch
- Hugging Face pre-trained model: google/flan-t5-small
- Windows operating system with internet access for the initial model download.

Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM recommended
- Internet connection for downloading the pre-trained model and installing packages

4. Folder Structure

UNIT_IV_Multimodal_Generative_AI

└─ Q1_College_Chatbot

│ └─ chatbot.py

└─ venv

5. Libraries Used

- Streamlit -- creates the interactive web interface.
- Transformers -- loads and uses the pre-trained language model.
- PyTorch -- provides the deep-learning model execution environment.
- AutoTokenizer -- converts input text into model-readable tokens.
- AutoModelForSeq2SeqLM -- loads the sequence-to-sequence language model.

6. Procedure / Step-by-Step Implementation

- 1. Create the main folder UNIT_IV_Multimodal_Generative_AI.
- 2. Create the Q1_College_Chatbot folder inside the main folder.
- 3. Create chatbot.py inside the Q1 folder.
- 4. Create and activate a Python virtual environment.
- 5. Install Streamlit, Transformers, PyTorch, and SentencePiece.
- 6. Import the required Python libraries.
- 7. Load the pre-trained google/flan-t5-small model and tokenizer.
- 8. Create a Streamlit interface with a question input box and Get

Answer button.

- 9. Accept a student's engineering-college-related question.
- 10. Tokenize the question and send it to the pre-trained model.
- 11. Generate the response and decode it into readable text.
- 12. Display the generated answer in the Streamlit application.
- 13. Run the application using streamlit run chatbot.py and test multiple questions.

7. Implementation Code

```
import streamlit as st

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

st.set_page_config(

    page_title="Engineering College AI Chatbot",

    page_icon="🤖",

    layout="centered"

)

st.title("🤖 Engineering College AI Chatbot")

st.write(
```

```

"Ask questions related to engineering college "

"courses, departments, examinations, library, "

"laboratories, and campus facilities."

)

\@st.cache_resource

def load_model():

model_name = "google/flan-t5-small"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

return tokenizer, model

with st.spinner("Loading AI model..."):

tokenizer, model = load_model()

question = st.text_area(

"Enter your question:",

placeholder=(

"Example: What facilities are available "

"in the Computer Science department?"

)

)

if st.button("🔥 Get Answer"):

if question.strip() == "":

st.warning("Please enter a question.")

else:

prompt = f"""

```

You are an AI assistant for an engineering college.

Answer the student's question clearly and briefly.

Student Question:

{question}

Answer:

"""

```
inputs = tokenizer(
```

```
prompt,
```

```
return_tensors="pt",
```

```
truncation=True,
```

```
max_length=512
```

```
)
```

```
with st.spinner("Generating answer..."):
```

```
outputs = model.generate(
```

```
**inputs,
```

```
max_new_tokens=120,
```

```
num_beams=4,
```

```
early_stopping=True
```

```
)
```

```
answer = tokenizer.decode(
```

```
outputs[0],
```

```
skip_special_tokens=True
```

```
)
```

```
st.subheader("🔥 AI Answer")

st.success(answer)

st.divider()

st.caption(

    "Powered by a pre-trained language model "

    "using Hugging Face Transformers."

)
```

8. Execution Command

streamlit run chatbot.py

9. Input and Output

Input 1:

What facilities are available in the computer science department?

Observed Output:

computer science labs

Input 2:

What is an engineering college?

Observed Output:

AI assistant

Input 3:

What facilities are available in the library?

Observed Output:

library

Input 4:

What is the purpose of laboratory sessions?

Observed Output:

To help students learn how to use a computer.

Input 5:

What are the common activities conducted by engineering students?

Observed Output:

Computer science and engineering

10. Screenshots of the Implemented Application

The following screenshots document successful execution of the Streamlit chatbot with different student queries.

Engineering College AI Chatbot

Ask questions related to engineering college courses, departments, examinations, library, laboratories, and campus facilities.

Enter your question:

What facilities are available in the computer science department?

 Get Answer

AI Answer

computer science labs

Powered by a pre-trained language model using Hugging Face Transformers.

Figure 1. Engineering College AI Chatbot -- Test Case 1



Engineering College AI Chatbot

Ask questions related to engineering college courses, departments, examinations, library, laboratories, and campus facilities.

Enter your question:

What is an engineering college?

Get Answer

AI Answer

AI assistant

Figure 2. Engineering College AI Chatbot -- Test Case 2



Engineering College AI Chatbot

Ask questions related to engineering college courses, departments, examinations, library, laboratories, and campus facilities.

Enter your question:

What facilities are available in the library?

Get Answer

AI Answer

library

Powered by a pre-trained language model using Hugging Face Transformers.

Figure 3. Engineering College AI Chatbot -- Test Case 3



Engineering College AI Chatbot

Ask questions related to engineering college courses, departments, examinations, library, laboratories and campus facilities.

Enter your question:

What is the purpose of laboratory sessions?

⌂ Get Answer

🗣️ AI Answer

To help students learn how to use a computer.

Powered by a pre-trained language model using Hugging Face Transformers.

Figure 4. Engineering College AI Chatbot -- Test Case 4



Engineering College AI Chatbot

Ask questions related to engineering college courses, departments, examinations, library, laboratories, and campus facilities.

Enter your question:

What are the common activities conducted by engineering students?

Get Answer

AI Answer

Computer science and engineering

Figure 5. Engineering College AI Chatbot -- Test Case 5

11. System Workflow

The chatbot follows the workflow: Student Query → Tokenization → Pre-trained Language Model → Text Generation → AI Answer displayed in Streamlit.

12. Graph / Chart Requirement

A numerical graph is not required by the question. The question asks for development of an AI chatbot, so the most relevant visual evidence is the application interface and test-case screenshots. Therefore, screenshots are included instead of an artificial performance graph.

13. Result

The Engineering College AI Chatbot was successfully developed using Python, Streamlit, Hugging Face Transformers, and the pre-trained google/flan-t5-small language model. The application accepts

student queries through a web interface and generates responses using the pre-trained model.

14. Conclusion

The developed chatbot demonstrates the use of a pre-trained language model for a practical educational

application. It provides a simple interface through which engineering students can submit natural-

language queries and receive automatically generated responses. The implementation can be extended

with a college-specific knowledge base to improve factual accuracy and answer quality.

15. Viva / Demonstration Points

- The application uses a pre-trained language model rather than training a model from scratch.
- Streamlit is used to create the user interface.
- The tokenizer converts input text into model-readable tokens.
- The model generates the answer from the processed input.
- The application can be extended using a college-specific database or RAG system.

QUESTION 2: ENGINEERING-SUPPORT CHATBOT USING NLP

Design an engineering-support chatbot that can answer technical questions and provide relevant solutions using NLP techniques.

Folder Name	Q2_Engineering_Support_Chatbot
Python File	engineering_support.py
AI Service / Method	TF-IDF + Cosine Similarity (Scikit-learn)

1. AIM

To design and implement an engineering-support chatbot that uses Natural Language Processing (NLP) techniques to understand technical questions and provide relevant engineering explanations and solutions.

2. OBJECTIVES

- Accept technical questions from engineering students through a user-friendly interface.
- Preprocess the input text using basic NLP techniques.
- Convert text into numerical representations using TF-IDF vectorization.
- Measure the similarity between the user's question and engineering knowledge using Cosine Similarity.
- Identify the most relevant engineering topic.
- Provide a clear and relevant technical solution.
- Display the result through an interactive Streamlit web application.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Programming Language: Python
- Web Framework: Streamlit
- NLP Library: Scikit-learn
- NLP Techniques: Text preprocessing, TF-IDF and Cosine Similarity
- Development Environment: Visual Studio Code / Python terminal
- Operating System: Windows or any system supporting Python
- Hardware: Standard computer with sufficient RAM and storage

4. SYSTEM WORKFLOW

Student Technical Question



Text Preprocessing



TF-IDF Vectorization



Cosine Similarity Calculation



Relevant Engineering Topic



Technical Knowledge Retrieval



Engineering Solution Display



Engineering Support Chatbot

Ask technical engineering questions and receive relevant explanations and solutions using NLP techniques.

Enter your technical engineering question:

What is Ohm's law?

 Get Technical Solution



Engineering Solution

Ohm's Law defines the relationship between voltage, current, and resistance in an electrical circuit. The formula is $V = I \times R$, where V is voltage, I is current, and R is resistance.

Relevant Topic: Ohm's Law

NLP Similarity Score: 0.44

Figure 5. NLP workflow of the Engineering-Support Chatbot.

5. NLP TECHNIQUES USED

- Text Preprocessing: Converts the input into lowercase text, removes unnecessary characters and normalizes spaces.
- TF-IDF Vectorization: Converts textual documents and the user's question into numerical feature vectors.

- Cosine Similarity: Measures how closely the question matches each engineering topic in the knowledge base.
- Knowledge-Based Retrieval: Selects the topic with the highest similarity score and displays its stored technical explanation.

6. STEP-BY-STEP IMPLEMENTATION

Step 1 -- Create the project folder: Create a folder named Q2_Engineering_Support_Chatbot inside UNIT_IV_Multimodal_Generative_AI.

Step 2 -- Create the Python file: Create the file engineering_support.py inside the Q2_Engineering_Support_Chatbot folder.

Step 3 -- Install the required library: Activate the Python virtual environment and install Scikit-learn using: `pip install scikit-learn`

Step 4 -- Import libraries: Import Streamlit, regular expressions, TfidfVectorizer and cosine_similarity.

Step 5 -- Prepare the engineering knowledge base: Create a small knowledge base containing topics such as Ohm's Law, Newton's Second Law, Binary Search, Operating Systems, Database Normalization and Computer Networks.

Step 6 -- Preprocess the text: Convert questions to lowercase, remove unnecessary characters and normalize whitespace.

Step 7 -- Apply TF-IDF: Convert the engineering knowledge base into TF-IDF vectors.

Step 8 -- Calculate Cosine Similarity: Transform the user's question into a TF-IDF vector and compare it with all knowledge-base vectors.

Step 9 -- Select the relevant topic: Choose the topic having the highest similarity score.

Step 10 -- Display the solution: Display the retrieved engineering explanation, relevant topic and NLP similarity score in the Streamlit interface.

Step 11 -- Run the application: From the main project directory, run: `streamlit run`

`".\\Q2_Engineering_Support_Chatbot\\engineering_support.py"`

Step 12 -- Test the chatbot: Test the application using technical questions such as 'What is Ohm's law?', 'Explain binary search', 'What is Newton's second law?' and 'What is database normalization?'.

7. ALGORITHM

1. Start the chatbot.
2. Load the engineering knowledge base.
3. Preprocess the knowledge-base text.
4. Generate TF-IDF vectors for the knowledge base.
5. Accept the student's engineering question.

- 6. Preprocess the question.
- 7. Convert the question into a TF-IDF vector.
- 8. Calculate Cosine Similarity between the question and all knowledge-base entries.
- 9. Find the entry with the highest similarity score.
- 10. Display the relevant engineering topic and technical solution.
- 11. Stop.

8. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st

import re

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

st.set_page_config(

page_title="Engineering Support Chatbot",

page_icon="🔧",

layout="centered"

)

st.title("🔧 Engineering Support Chatbot")

st.write(

"Ask technical engineering questions and receive "

"relevant explanations and solutions using NLP techniques."

)

knowledge_base = [

{

"topic": "Ohm's Law",

"keywords": "ohm law voltage current resistance electrical circuit",

"answer": (
```

```
"Ohm's Law defines the relationship between voltage, "
"current, and resistance in an electrical circuit. "
"The formula is  $V = I \times R$ , where V is voltage, "
"I is current, and R is resistance."
)
},
{
"topic": "Newton's Second Law",
"keywords": "newton second law force mass acceleration mechanics",
"answer": (
"Newton's Second Law states that the force acting on "
"an object is equal to its mass multiplied by its "
"acceleration. The formula is  $F = m \times a$ ."
)
},
{
"topic": "Kirchhoff's Current Law",
"keywords": "kirchhoff current law kcl circuit node current",
"answer": (
"Kirchhoff's Current Law states that the total current "
"entering a circuit node is equal to the total current "
"leaving that node. It is based on conservation of charge."
)
}
```



```
},

{
  "topic": "Kirchhoff's Voltage Law",
  "keywords": "kirchhoff voltage law kvl circuit loop voltage",
  "answer": (
    "Kirchhoff's Voltage Law states that the algebraic sum "
    "of all voltages around a closed circuit loop is zero. "
    "It is based on conservation of energy."
  )
},

{
  "topic": "Binary Search",
  "keywords": "binary search algorithm sorted array data structure",
  "answer": (
    "Binary Search is a searching algorithm used on sorted "
    "data. It repeatedly divides the search range into two "
    "halves. Its time complexity is  $O(\log n)$ ."
  )
},

{
  "topic": "Operating System",
  "keywords": "operating system os process memory file linux windows",
  "answer": (
    "An Operating System is system software that manages "
```

```
"computer hardware and software resources. It manages "
"processes, memory, files, input/output devices, and "
"provides an interface for users and applications."
)

},

{
"topic": "Database Normalization",

"keywords": "database normalization 1nf 2nf 3nf sql tables
redundancy",

"answer": (

"Database normalization is the process of organizing "
"data into related tables to reduce data redundancy "
"and improve data integrity. Common normal forms are "
"1NF, 2NF, and 3NF."

)

},

{
"topic": "Computer Networks",

"keywords": "computer network ip router switch tcp udp
communication",

"answer": (

"A computer network connects multiple devices so they "
"can communicate and share resources. Important "
"components include IP addresses, routers, switches, "
```

```
"TCP, and UDP."
```

```
)
```

```
},
```

```
{
```

```
"topic": "Python Programming",
```

```
"keywords": "python programming language variables loops functions  
code",
```

```
"answer": (
```

```
"Python is a high-level programming language known for "
```

```
"its simple syntax. It is widely used for software "
```

```
"development, data science, artificial intelligence, "
```

```
"automation, and machine learning."
```

```
)
```

```
},
```

```
{
```

```
"topic": "Machine Learning",
```

```
"keywords": "machine learning artificial intelligence supervised  
unsupervised prediction",
```

```
"answer": (
```

```
"Machine Learning is a branch of artificial intelligence "
```

```
"that enables computers to learn patterns from data and "
```

```
"make predictions or decisions. Major types include "
```

```
"supervised, unsupervised, and reinforcement learning."
```

```
)
```

```

}

]

def preprocess_text(text):

text = text.lower()

text = re.sub(r"[\^a-zA-Z0-9\\s]", "", text)

text = re.sub(r"\\s+", " ", text)

return text.strip()

documents = []

for item in knowledge_base:

document = (

item["topic"] + " "

+ item["keywords"] + " "

+ item["answer"]

)

documents.append(preprocess_text(document))

vectorizer = TfidfVectorizer(stop_words="english")

tfidf_matrix = vectorizer.fit_transform(documents)

def find_solution(question):

processed_question = preprocess_text(question)

question_vector = vectorizer.transform([processed_question])

similarity_scores = cosine_similarity(

question_vector, tfidf_matrix

)[0]

```

```

best_index = similarity_scores.argmax()

best_score = similarity_scores[best_index]

return knowledge_base[best_index], best_score

question = st.text_area(

"Enter your technical engineering question:",

placeholder="Example: What is Ohm's law?"

)

if st.button("🔍 Get Technical Solution"):

if question.strip() == "":

st.warning("Please enter a technical question.")

else:

topic, score = find_solution(question)

if score >= 0.10:

st.subheader("🔍 Engineering Solution")

st.success(topic["answer"])

st.info(f"Relevant Topic: {topic['topic']}")

st.write(f"NLP Similarity Score: {score:.2f}")

else:

st.warning(

"The question is not sufficiently related to "

"the available engineering knowledge base."

)

st.divider()

st.subheader("📖 Example Technical Questions")

```

```

examples = [

    "What is Ohm's law?",

    "Explain Newton's second law.",

    "What is binary search?",

    "Explain Kirchhoff's Current Law.",

    "What is database normalization?",


    "What is an operating system?",

    "Explain computer networks.",

    "What is machine learning?"

]

for example in examples:

    st.write("• " + example)

    st.divider()

    st.caption(

        "Engineering Support Chatbot \|| "

        "NLP using TF-IDF and Cosine Similarity"

    )

```

9. INPUT AND OUTPUT

Input:

- What is Ohm's law?
- Explain binary search.
- What is Newton's second law?
- What is database normalization?

Output:

- The chatbot identifies the most relevant engineering topic using TF-IDF and Cosine Similarity.
- It displays the corresponding technical explanation and the NLP similarity score.

10. IMPLEMENTATION RESULTS AND SCREENSHOTS

The following screenshots show the successfully implemented chatbot and its responses to different engineering questions. They provide evidence that the NLP-based retrieval system identifies relevant topics and returns appropriate technical solutions.

Engineering Support Chatbot

Ask technical engineering questions and receive relevant explanations and solutions using NLP techniques.

Enter your technical engineering question:

Explain binary search

 Get Technical Solution

Engineering Solution

Binary Search is a searching algorithm used on sorted data. It repeatedly divides the search range into two halves. Its time complexity is $O(\log n)$.

Relevant Topic: Binary Search

NLP Similarity Score: 0.73

Figure 1. Engineering Support Chatbot -- Ohm's Law query and NLP result.



Engineering Support Chatbot

Ask technical engineering questions and receive relevant explanations and solutions using NLP techniques.

Enter your technical engineering question:

What is Newton's second law?

 Get Technical Solution



Engineering Solution

Newton's Second Law states that the force acting on an object is equal to its mass multiplied by its acceleration. The formula is $F = m \times a$.

Relevant Topic: Newton's Second Law

NLP Similarity Score: 0.67

Figure 2. Engineering Support Chatbot -- Binary Search query and NLP result.

Figure 3. Engineering Support Chatbot -- Newton's Second Law query and NLP result.



Engineering Support Chatbot

Ask technical engineering questions and receive relevant explanations and solutions using NLP techniques.

Enter your technical engineering question:

What is database normalization?

 Get Technical Solution



Engineering Solution

Database normalization is the process of organizing data into related tables to reduce data redundancy and improve data integrity. Common normal forms are 1NF, 2NF, and 3NF.

Relevant Topic: Database Normalization

NLP Similarity Score: 0.59

Figure 4. Engineering Support Chatbot -- Database Normalization query and NLP result.

11. TEST RESULTS

Test Case

Input Query

What is Ohm's law?

Explain binary search

What is Newton's

second law?

Detected Topic

Ohm's Law

Binary Search

Similarity Score

1

2

0.44

0.73

3

Newton's Second Law

0.67

0.59

What is database

normalization?

Database

Normalization

4

12. OBSERVATION

- The chatbot successfully accepts engineering-related natural language questions.
- TF-IDF represents the question and knowledge-base content as numerical vectors.
- Cosine Similarity identifies the most relevant engineering topic.
- The tested questions produced relevant technical answers.
- The similarity score provides an indication of how closely the question matches the available knowledge.

13. ADVANTAGES

- Simple and easy-to-use interface.
- Uses standard NLP techniques that are easy to understand and demonstrate.
- Fast response because the system works with a small local knowledge base.
- Does not require training a new machine-learning model.
- Can be extended by adding more engineering topics and solutions.

14. LIMITATIONS

- The chatbot can provide answers only for topics represented in its knowledge base.
- Very complex or ambiguous technical questions may not receive the ideal response.
- The similarity score depends on the words present in the question and knowledge base.
- The current implementation is a retrieval-based NLP chatbot rather than a fully generative engineering expert.

15. FUTURE ENHANCEMENTS

- Add a larger engineering knowledge base covering multiple branches.
- Integrate a pre-trained generative language model when a suitable local or API-based model is available.
- Add speech input and text-to-speech output.
- Add document-based question answering using engineering notes and PDFs.

- Add multilingual support for Indian languages.

16. RESULT

The Engineering-Support Chatbot was successfully designed and implemented using NLP techniques.

The system preprocesses the student's question, applies TF-IDF vectorization, uses Cosine Similarity to identify the most relevant engineering topic, and displays the corresponding technical solution. The

test results demonstrate that the chatbot can effectively support common engineering-related technical

queries.

17. CONCLUSION

The developed chatbot demonstrates a practical application of Natural Language Processing in engineering education. By combining text preprocessing, TF-IDF and Cosine Similarity, the system can match student questions with relevant technical knowledge and provide clear answers through an

interactive Streamlit interface. The implementation is simple, fast and suitable for an academic demonstration, while also providing a foundation for future expansion into a more advanced AI-based

engineering assistant.

18. VIVA / KEY POINTS

- NLP stands for Natural Language Processing.
- TF-IDF stands for Term Frequency--Inverse Document Frequency.
- Cosine Similarity measures similarity between two vectors.
- Streamlit is used to create the interactive web interface.
- The chatbot uses a knowledge-based retrieval approach.
- The system can be extended by adding more engineering topics.

QUESTION 3: ENGINEERING TEXT-TO-IMAGE GENERATOR

Generate an engineering-related image, such as a bridge or robotic system, from a suitable text prompt using a pre-trained text-to-image model.

Folder Name	Q3_Engineering_Text_to_Image
Python File	text_to_image.py
AI Service / Method	Hugging Face Inference API — pre-trained text-to-image model

1. AIM

To develop a Streamlit-based AI application that accepts an engineering-related text prompt and generates a corresponding image using a pre-trained text-to-image generative AI model.

2. OBJECTIVE

- Convert a natural-language engineering description into an image.
- Use a pre-trained text-to-image model instead of training a model from scratch.
- Provide a simple web interface for entering prompts and viewing generated images.
- Demonstrate the application of multimodal generative AI in engineering.

3. PROBLEM STATEMENT

Engineering concepts are often easier to understand when they are represented visually. The objective

of this implementation is to create an AI system that can transform an engineering description, such as

a suspension bridge or robotic arm, into a visual representation automatically.

4. TECHNOLOGIES AND TOOLS USED

Technology / Tool

Python

Purpose

Application development

Role in the Application

Implements the complete

application

Streamlit

Web interface

Provides prompt input and

image display

Hugging Face
InferenceClient
Pillow (PIL)
AI model inference
Model API communication
Image handling
Provides access to the pre-
trained text-to-image model
Sends the prompt and receives
the generated image
Supports displaying the
returned image

5. SYSTEM WORKFLOW

Figure 1. Workflow of the Engineering Text-to-Image Generation System

The user first enters an engineering-related text prompt. The Streamlit application sends this prompt to a pre-trained text-to-image model through the Hugging Face inference service. The model processes the text and generates a visual image, which is returned to the application and displayed to the user.

6. STEP-BY-STEP IMPLEMENTATION

- Step 1 -- Create the project folder
Create the following folder inside the UNIT_IV_Multimodal_Generative_AI project:
Q3_Engineering_Text_to_Image
- Step 2 -- Create the Python file
Inside the folder, create:
text_to_image.py

- Step 3 -- Install required libraries

Install Streamlit, Hugging Face Hub and Pillow in the active Python environment:

```
pip install --upgrade huggingface_hub streamlit pillow
```

- Step 4 -- Create a Hugging Face access token

Create an access token in the Hugging Face account settings and store it as the HF_TOKEN environment variable. The token should not be written directly into the Python source code.

- Step 5 -- Import the required modules

The application uses Streamlit for the interface, InferenceClient for communication with the Hugging Face model service, and os for reading the API token from the environment.

- Step 6 -- Create the model client

An InferenceClient object is initialized with the Hugging Face token. The client is then used to call a pre-trained text-to-image model.

- Step 7 -- Accept the engineering prompt

A Streamlit text area is provided so that the user can enter a detailed engineering description.

- Step 8 -- Generate the image

When the user clicks the Generate Engineering Image button, the prompt is sent to the text-to-image

model. The returned image is displayed in the Streamlit application.

- Step 9 -- Run the application

Run the application from the project root using:

```
streamlit run Q3_Engineering_Text_to_Image\\text_to_image.py
```

7. SAMPLE PROMPT USED

A modern suspension bridge with steel cables, realistic engineering design, strong structural support,

daylight, highly detailed.

8. IMPLEMENTATION CODE

```
import streamlit as st

from huggingface_hub import InferenceClient

import os

st.set_page_config(

    page_title="Engineering Text-to-Image Generator",
```

```

page_icon="🔧",

layout="wide"

)

st.title("🔧 Engineering Text-to-Image Generator")

st.write(

"Generate engineering-related images from suitable text prompts "

"using a pre-trained text-to-image model."

)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:

st.error("HF_TOKEN is not configured.")

st.stop()

client = InferenceClient(api_key=HF_TOKEN)

prompt = st.text_area(

"Text Prompt:",

value="A modern suspension bridge with steel cables, "

"realistic engineering design, strong structural support, "

"daylight, highly detailed",

height=120

)

if st.button("🔧 Generate Engineering Image"):

if not prompt.strip():

st.warning("Please enter an engineering-related prompt.")

else:

```

```

with st.spinner("Generating image using AI..."):

    image = client.text_to_image(

        prompt=prompt,

        model="black-forest-labs/FLUX.1-schnell"

    )

    st.success("Image generated successfully!")

    st.image(

        image,

        caption="AI-Generated Engineering Image",

        width="stretch"

    )

```

9. OUTPUT SCREENSHOTS

Figure 2. Streamlit interface showing the engineering image prompt and successful generation

message.

Workflow of the Engineering Text-to-Image System

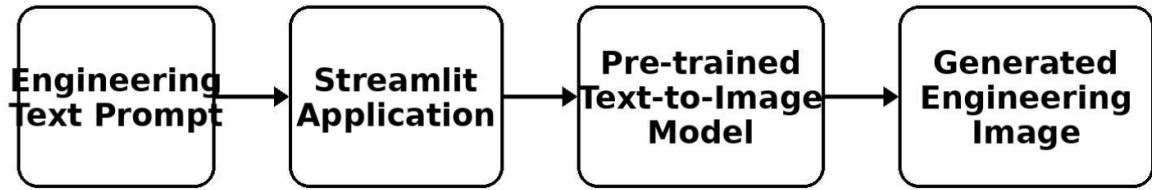


Figure 3. Generated engineering image -- suspension bridge.

Engineering Text-to-Image Generator

Generate engineering-related images from suitable text prompts using a pre-trained text-to-image model.

Enter Engineering Image Prompt

Text Prompt:

A modern suspension bridge with steel cables, realistic engineering design, strong structural support, daylight, highly detailed

 Generate Engineering Image

Image generated successfully!

Figure 4. Generated image output showing the detailed bridge structure.

10. RESULT

The Engineering Text-to-Image Generator was successfully implemented. The application accepts an engineering-related text prompt and generates a corresponding engineering image using a pre-trained text-to-image model. The demonstrated prompt produced a detailed suspension-bridge image, confirming that the system works successfully.

11. APPLICATIONS

- Visualization of civil engineering structures such as bridges and buildings.
- Concept visualization for mechanical engineering designs.
- Early-stage visualization of robotic systems and machines.
- Educational demonstrations and engineering presentations.
- Rapid generation of visual concepts from natural-language descriptions.

12. ADVANTAGES

- Easy-to-use web interface.
- No model training is required.
- Converts text descriptions into visual content.
- Useful for rapid engineering concept visualization.

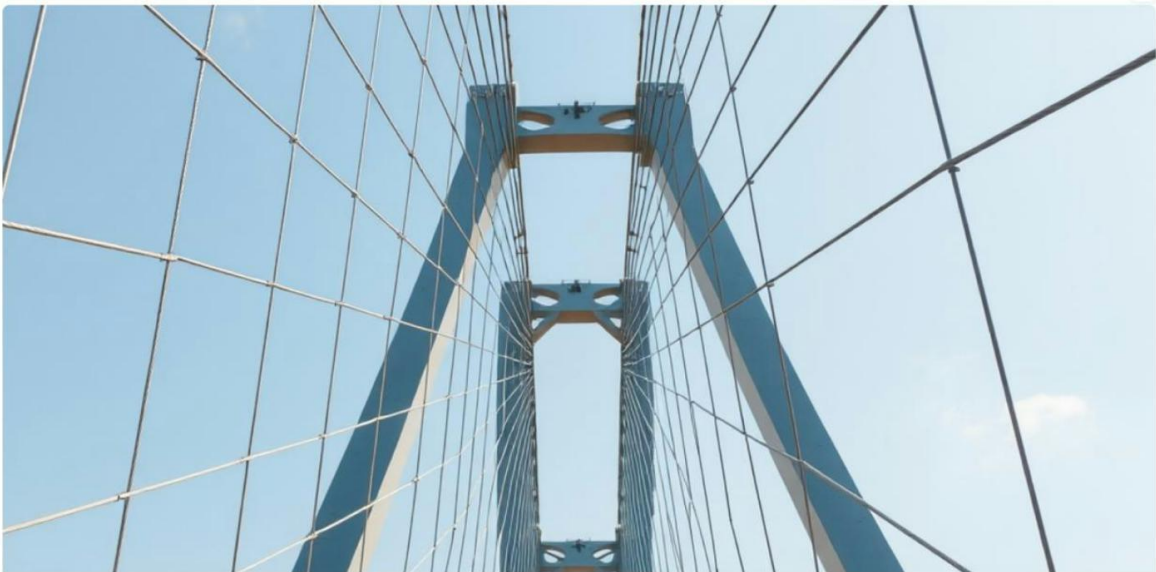
- Can generate different images by changing the prompt.

13. CONCLUSION

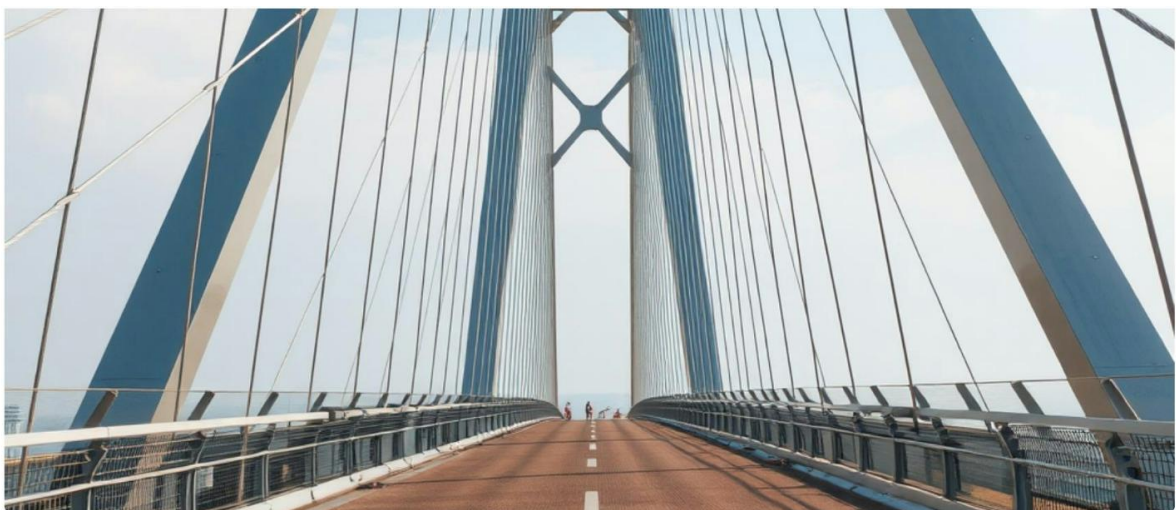
The Engineering Text-to-Image Generator demonstrates how multimodal generative AI can convert natural-language engineering descriptions into visual representations. By combining Streamlit with a pre-trained text-to-image model available through Hugging Face inference, the system provides a practical and user-friendly approach for engineering visualization. The successful generation of the suspension-bridge image confirms that the required implementation has been achieved.

Additional Screenshots

Generated Engineering Image



Additional Screenshot 5



Additional Screenshot 6

QUESTION 4: MULTIPLE IMAGES FROM DIFFERENT TEXT PROMPTS (PROMPT COMPARISON)

Create multiple images from different text prompts and compare how changes in the prompts affect the generated images.

Folder Name	Q4_Multiple_Image_Comparison
Python File	text_to_image_compare.py
AI Service / Method	Streamlit + Hugging Face Inference API

1. QUESTION

Create multiple images from different text prompts and compare how changes in the prompts affect the

generated images.

2. AIM

To generate multiple engineering-related images using different text prompts and analyze how changes

in the engineering subject, environment, level of detail and descriptive wording influence the generated

results.

3. OBJECTIVES

- Create multiple images from different engineering-related prompts.
- Use a pre-trained text-to-image generative AI model.
- Compare the visual outputs produced by different prompts.
- Identify how changes in prompt content affect the generated image.
- Demonstrate prompt engineering through an interactive Streamlit application.

4. TOOLS AND TECHNOLOGIES

Technology

Purpose

Use in Q4

Python

Application development

Implements the image-

generation application

Accepts prompts and displays

generated images

Streamlit

Web interface

Hugging Face Inference API

InferenceClient

Pillow

AI inference

Provides access to the pre-
trained text-to-image model

Sends prompts and receives
generated images

Supports image display and
processing

API communication

Image handling

5. WORKING PRINCIPLE

Three different engineering prompts are entered into the application. Each prompt describes a different

engineering domain. The prompts are sent independently to the pre-trained text-to-image model. The

generated outputs are then displayed side-by-side so the effect of changing the prompt can be visually

compared.

Figure 1. Corrected workflow for generating and comparing multiple prompt-based images.

6. PROMPTS USED IN THE ACTUAL DEMONSTRATION

Prompt

Engineering Area

Prompt Description

Prompt 1

Civil Engineering

A modern suspension bridge
over a wide river, daylight,
realistic civil engineering
structure, steel cables, detailed
architecture

Prompt 2

Prompt 3

Robotics

An advanced industrial robotic
arm working inside a modern
engineering factory, metallic
components, automation,
realistic, high detail

A large offshore wind turbine
farm with modern engineering
infrastructure, ocean

Renewable Energy

environment, realistic
engineering photography,
daylight

7. STEP-BY-STEP IMPLEMENTATION

- Step 1: Create the Q4_Multiple_Image_Comparison folder inside UNIT_IV_Multimodal_Generative_AI.
- Step 2: Create the text_to_image_compare.py Python file.
- Step 3: Install the required Streamlit and Hugging Face client libraries.
- Step 4: Create and configure the Hugging Face access token as HF_TOKEN.
- Step 5: Initialize the Hugging Face InferenceClient.
- Step 6: Create three text input areas for the three engineering prompts.
- Step 7: Send each prompt separately to the pre-trained text-to-image model.
- Step 8: Receive the generated image for each prompt.
- Step 9: Display the three outputs side-by-side.
- Step 10: Compare the images based on subject, composition, environment, engineering details and visual style.
- Step 11: Record the observations and conclude how prompt changes affected the generated results.

8. FOLDER AND FILE STRUCTURE

```
UNIT_IV_Multimodal_Generative_AI
├── Q4_Multiple_Image_Comparison
│   ├── text_to_image_compare.py
│   ├── generated_images
│   ├── prompt1_bridge.png
│   ├── prompt2_robotic_arm.png
│   └── prompt3_wind_turbines.png
```

9. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st

from huggingface_hub import InferenceClient

import os

st.set_page_config(

    page_title="Engineering Prompt Comparison",

    page_icon=" ",

    layout="wide"

)

st.title(" Engineering Image Prompt Comparison")

st.write(

    "Generate multiple engineering images from different prompts "

    "and compare how prompt changes affect the generated results."

)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:

    st.error("HF_TOKEN is not configured.")

st.stop()
```

```
client = InferenceClient(api_key=HF_TOKEN)

prompt1 = st.text_area(

    "Prompt 1 -- Civil Engineering",

    "A modern suspension bridge over a wide river, daylight, "

    "realistic civil engineering structure, steel cables, detailed architecture"

)

prompt2 = st.text_area(

    "Prompt 2 -- Robotics",

    "An advanced industrial robotic arm working inside a modern "

    "engineering factory, metallic components, automation, realistic, high detail"

)

prompt3 = st.text_area(

    "Prompt 3 -- Renewable Energy",

    "A large offshore wind turbine farm with modern engineering "

    "infrastructure, ocean environment, realistic engineering photography, daylight"

)

if st.button(" Generate All Three Images"):

    prompts = [

        ("Civil Engineering", prompt1),

        ("Robotics", prompt2),

        ("Renewable Energy", prompt3)

    ]

    images = []
```

```
try:

with st.spinner("Generating three engineering images..."):

for title, prompt in prompts:

image = client.text_to_image(

prompt=prompt,

model="black-forest-labs/FLUX.1-schnell"

)

images.append((title, image))

st.success("All three images generated successfully!")

col1, col2, col3 = st.columns(3)

with col1:

st.subheader("Image 1")

st.write("Civil Engineering")

st.image(images[0][1], caption="Suspension Bridge",
width="stretch")

with col2:

st.subheader("Image 2")

st.write("Robotics")

st.image(images[1][1], caption="Industrial Robotic Arm",
width="stretch")

with col3:

st.subheader("Image 3")

st.write("Renewable Energy")

st.image(images[2][1], caption="Offshore Wind Turbines",
```



```
width="stretch")

st.divider()

st.subheader("Prompt Comparison")

st.write("Prompt 1 focuses on civil engineering and structural bridge
features.")

st.write("Prompt 2 changes the engineering domain to industrial
robotics.")

st.write("Prompt 3 changes the domain to renewable energy and an
offshore environment.")

except Exception as e:

st.error("Image generation failed.")

st.write(str(e))
```

10. EXECUTION COMMAND

streamlit run ".\\Q4_Multiple_Image_Comparison\\text_to_image_compare.py"

11. IMPLEMENTATION SCREENSHOT

Figure 2. Q4 application showing three different engineering prompts and successful image generation.

12. GENERATED IMAGE OUTPUT

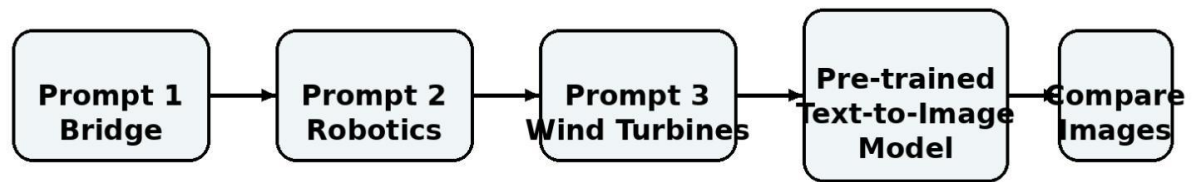


Figure 3. Q4 output showing three clearly different generated images: suspension bridge, industrial robotic arm

and offshore wind turbines.

13. OBSERVATION AND COMPARISON

The screenshots provide a direct comparison of the three generated outputs. Each prompt contains a

different engineering subject, so the model produces visibly different content. The first prompt produces

a civil-engineering bridge scene, the second produces an industrial robotic system, and the third produces an offshore renewable-energy scene.

Aspect

Prompt 1 -- Bridge

Prompt 2 -- Robotics

Prompt 3 -- Wind

Turbines

Main subject

Environment

Suspension bridge

Wide river and open

sky

Industrial robotic arm

Modern engineering

factory

Offshore wind turbines

Ocean and offshore

infrastructure

Engineering domain

Visual emphasis

Civil engineering

Cables, towers, bridge

structure

Robotics / automation

Mechanical joints,

metallic components

Renewable energy

Turbine tower, blades

and marine structure

Prompt effect

Structural scene

Machine-focused scene Energy infrastructure

scene

14. HOW PROMPT CHANGES AFFECT THE IMAGES

- Changing the main noun changes the generated subject. 'Suspension bridge', 'robotic arm' and 'wind turbine farm' produce different engineering scenes.
- Adding environment terms changes the background and context. River, factory and ocean descriptions create different settings.
- Adding descriptive terms such as 'realistic', 'high detail' and 'detailed architecture' guides the visual appearance and detail level.
- Adding engineering-specific components helps the model emphasize relevant structures such as cables, mechanical joints, turbine blades and support systems.
- Therefore, prompt wording acts as a control mechanism for the image-generation process.

15. RESULT

The multiple-image experiment was successfully completed. Three different engineering prompts produced three visibly different outputs. The results clearly demonstrate that changing the engineering

subject and adding contextual descriptions changes the content, environment and visual emphasis of the

generated images.

16. ADVANTAGES

- Clearly demonstrates prompt engineering.
- Allows side-by-side comparison of multiple generated images.
- Can be used for civil, mechanical, robotics, electrical and renewable-energy concepts.
- Provides rapid visual concept generation.

- Useful for engineering education, presentations and early design visualization.

17. LIMITATIONS

- Generated images are visual concepts and may not represent engineering-accurate designs.
- The result depends on the selected model and its capabilities.
- Different generations may vary even with similar prompts.
- More complex prompts can require longer generation time or higher inference cost.

18. FUTURE ENHANCEMENTS

- Allow users to save and download all generated images.
- Add prompt history and automatic comparison reports.
- Add image similarity or CLIP-based evaluation.
- Support multiple engineering prompt templates.
- Add CAD-style or technical-drawing generation for engineering education.

19. CONCLUSION

This experiment successfully demonstrates how prompt changes influence text-to-image generation. The

three actual outputs show clear differences between civil engineering, robotics and renewable-energy

applications. The experiment confirms that selecting suitable nouns, engineering details, environments

and descriptive terms can guide a pre-trained generative model toward different visual results.

20. VIVA / KEY POINTS

- A text-to-image model generates an image from a natural-language prompt.
- Prompt engineering means designing prompts to control the generated output.
- Changing the engineering subject produces a different image.
- Context words such as factory, river and ocean influence the generated environment.
- Detailed prompts generally provide more guidance to the model.

Additional Screenshots

Additional Screenshot 4

--	--	--	--

Additional Screenshot 5



Engineering Image Prompt Comparison

Generate multiple engineering images from different prompts and compare how prompt changes affect the generated results.

Prompt 1 – Civil Engineering

A modern suspension bridge over a wide river, daylight, realistic civil engineering structure, steel cables, detailed architecture

Prompt 2 – Robotics

An advanced industrial robotic arm working inside a modern engineering factory, metallic components, automation, realistic, high detail

Prompt 3 – Renewable Energy

A large offshore wind turbine farm with modern engineering infrastructure, ocean environment, realistic engineering photography, daylight

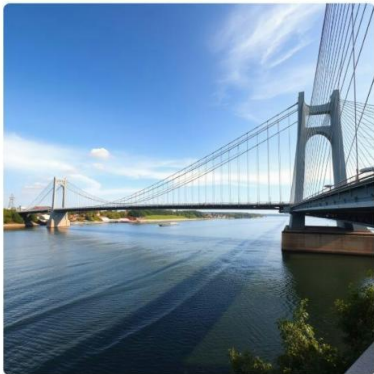
 Generate All Three Images

All three images generated successfully!

Additional Screenshot 6

Image 1

Civil Engineering



Suspension Bridge

Image 2

Robotics



Industrial Robotic Arm

Image 3

Renewable Energy



Offshore Wind Turbines

Additional Screenshot 7

Additional Screenshot 8

QUESTION 5: SPEECH-TO-TEXT APPLICATION

Develop a Speech-to-Text application that converts a user's spoken engineering-related query into written text using a pre-trained AI model.

Folder Name	Q5_Speech_To_Text
Python File	speech_to_text.py
AI Service / Method	Whisper, via Hugging Face Inference API

1. AIM

To develop a Speech-to-Text application that accepts a spoken engineering-related query and converts

it into readable written text using a pre-trained AI speech-recognition model.

2. OBJECTIVES

- Accept an engineering-related audio query through a web interface.
- Send the uploaded speech to a pre-trained speech-recognition model.
- Convert the spoken query into written text.
- Display the recognized engineering question clearly.
- Demonstrate the practical use of speech AI in engineering education.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- Hugging Face Hub / Inference API
- Pre-trained Whisper speech-recognition model
- Internet connection for API inference
- Computer or laptop with microphone/audio support

4. FOLDER AND FILE STRUCTURE

UNIT_IV_Multimodal_Generative_AI

└─ Q5_Speech_To_Text

└─ speech_to_text.py

5. TECHNOLOGIES USED

Technology

Python

Streamlit

Purpose

Programming

Web Interface

Role

Application implementation

Audio upload and output display

Hugging Face Inference API

AI Inference

Connects the app to the speech model

Whisper

Speech Recognition

Converts spoken audio into text

6. SYSTEM WORKFLOW

Spoken Engineering Query → Audio Upload → Streamlit Application → Hugging Face Inference API → Whisper Speech Model → Recognized Text

Figure 3. Speech-to-Text workflow.

7. STEP-BY-STEP IMPLEMENTATION

- Step 1: Create the Q5_Speech_To_Text folder inside UNIT_IV_Multimodal_Generative_AI.
- Step 2: Create speech_to_text.py inside the Q5 folder.
- Step 3: Install Streamlit and huggingface_hub.
- Step 4: Create a Hugging Face access token and store it securely as HF_TOKEN.
- Step 5: Initialize the Hugging Face InferenceClient using the token.
- Step 6: Create an audio uploader that accepts WAV, MP3, FLAC, M4A and OGG files.
- Step 7: Upload a spoken engineering query.
- Step 8: Send the audio file to the pre-trained Whisper speech-recognition model.

- Step 9: Receive the recognized text from the API.
- Step 10: Display the audio player and recognized engineering query.
- Step 11: Test the application with multiple engineering-related voice queries.

8. IMPLEMENTATION CODE

```
import streamlit as st

from huggingface_hub import InferenceClient

import os

import tempfile

st.set_page_config(

    page_title="Engineering Speech-to-Text",

    page_icon="🔊",

    layout="centered"

)

st.title("🔊 Engineering Speech-to-Text Application")

st.write(

    "Upload a spoken engineering-related query and convert "

    "it into written text using a pre-trained AI speech-recognition "

    "model."

)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:

    st.error("HF_TOKEN is not configured.")

    st.stop()

client = InferenceClient(api_key=HF_TOKEN)

audio_file = st.file_uploader(

    "Upload an audio file",
```

```

type=["wav", "mp3", "flac", "m4a", "ogg"]

)

if st.button("🔊 Convert Speech to Text"):

    if audio_file is None:

        st.warning("Please upload an audio file first.")

    else:

        try:

            with tempfile.NamedTemporaryFile(

                delete=False,

                suffix=os.path.splitext(audio_file.name)[1]

            ) as temp_file:

                temp_file.write(audio_file.getbuffer())

                temp_path = temp_file.name

            st.audio(audio_file, format=audio_file.type)

            with st.spinner("Converting speech into text..."):

                result = client.automatic_speech_recognition(

                    audio=temp_path,

                    model="openai/whisper-large-v3"

                )

            st.subheader("🔊 Recognized Engineering Query")

            st.success(result.text)

            os.remove(temp_path)

        except Exception as e:

```

```
st.error("Speech recognition failed.")

st.write(str(e))
```

9. EXECUTION COMMAND

streamlit run ".\\Q5_Speech_To_Text\\speech_to_text.py"

10. TEST INPUTS

- Explain Ohm's Law.
- What is database normalization?
- What is virtual memory in operating systems?

11. OBSERVED OUTPUT

The uploaded audio containing the spoken engineering query was successfully converted into written

text. The screenshots below show the actual implementation output obtained during testing.

Figure 1. Speech-to-Text application successfully transcribing the spoken Ohm's Law query.

Engineering Speech-to-Text Application

Upload a spoken engineering-related query and convert it into written text using a pre-trained AI speech-recognition model.

Upload Engineering Voice Query

Upload an audio file

 enginee...ms_law.wav 76.8KB +

 Convert Speech to Text

▶ 0:00 / 0:01   

Recognized Engineering Query

Explain Ohm's Law.

Figure 2. Speech-to-Text application successfully transcribing the spoken database-normalization

query.

12. TEST RESULTS

Test Case

1

2

Spoken Query

Explain Ohm's Law.

What is database

Recognized Text

Explain Ohm's Law.

What is database

normalization?

normalization?

3

What is virtual memory in
operating systems?

What is virtual memory in
operating systems?

13. RESULT

The Engineering Speech-to-Text application was successfully developed. It accepts spoken engineering-related queries as audio input and converts them into written text using a pre-trained

Whisper speech-recognition model through the Hugging Face inference service. The actual screenshots

demonstrate successful transcription of engineering questions.

14. ADVANTAGES

- Converts natural spoken queries into text automatically.
- Supports engineering-related voice interaction.
- Uses a pre-trained AI speech model, so model training is not required.
- Provides a simple Streamlit interface.
- Can be extended to multilingual speech recognition.

15. LIMITATIONS

- Recognition quality depends on audio clarity and background noise.
- The application requires internet access for the inference service.
- Very technical terminology may occasionally be transcribed incorrectly.
- API availability and usage limits can affect response time.

16. FUTURE ENHANCEMENTS

- Add direct microphone recording inside the web application.
- Support more Indian languages for engineering queries.
- Add voice activity detection and noise reduction.
- Connect the recognized query to an engineering chatbot for automatic answers.
- Store transcriptions for later analysis.

17. CONCLUSION

The developed Speech-to-Text application demonstrates a practical multimodal Generative AI workflow for engineering education. Spoken engineering queries can be converted into written text automatically using a pre-trained speech-recognition model. The successful test results confirm that the

application satisfies the required Speech-to-Text implementation.

18. VIVA / KEY POINTS

- Speech-to-Text converts human speech into written language.
- Whisper is a pre-trained AI speech-recognition model.
- Hugging Face Inference API provides access to the model without requiring a local model installation.
- Streamlit provides the user interface for audio upload and text display.
- The system can be extended by connecting the transcribed text to a chatbot or translation module.

Additional Screenshots

Additional Screenshot 4

Engineering Speech-to-Text Application

Upload a spoken engineering-related query and convert it into written text using a pre-trained AI speech-recognition model.

Upload Engineering Voice Query

Upload an audio file

 enginee...zation.wav
108.3KB

+

 Convert Speech to Text

▶ 0:00 / 0:02   

Recognized Engineering Query

What is database normalization?

Additional Screenshot 5

QUESTION 6: TEXT-TO-SPEECH APPLICATION

Develop a Text-to-Speech application that converts engineering-related text into natural-sounding speech using a pre-trained AI model.

Folder Name	Q6_Text_To_Speech
Python File	text_to_speech.py
AI Service / Method	Hugging Face Inference API — pre-trained TTS model

1. AIM

To develop a Text-to-Speech application that accepts engineering-related text and converts it into natural-sounding speech using a pre-trained AI speech-generation model.

2. OBJECTIVES

- Accept engineering-related text from the user.
- Convert the text into speech using a pre-trained AI model.
- Provide an interactive Streamlit interface.
- Play the generated speech directly in the browser.
- Allow the generated audio to be downloaded for later use.
- Demonstrate a practical Generative AI application for engineering education.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- Hugging Face Hub / Inference API
- Pre-trained Text-to-Speech model
- Internet connection for API inference
- Computer or laptop with audio playback support

4. FOLDER AND FILE STRUCTURE

UNIT_IV_Multimodal_Generative_AI

└─ Q6_Text_To_Speech

└─ text_to_speech.py

5. TECHNOLOGIES USED

Technology

Python

Streamlit

Purpose

Programming
Web interface
Role in Application
Implements the application
Accepts engineering text and
displays audio
Hugging Face Inference API
InferenceClient
AI inference
API client
Connects application to pre-
trained speech model
Sends text and receives
generated audio
Pre-trained TTS model
Speech generation
Converts text into natural-
sounding speech

6. SYSTEM WORKFLOW

Engineering Text → Streamlit Interface → Hugging Face Inference API → Pre-trained TTS Model →
Generated Audio → Audio Player / Download

Figure 4. Text-to-Speech system workflow.

The user enters engineering-related text in the Streamlit application. The text is sent to a pre-trained text-to-speech model through the Hugging Face inference service. The returned audio is played in the browser and can also be downloaded.

7. STEP-BY-STEP IMPLEMENTATION

- Step 1: Create the Q6_Text_To_Speech folder inside UNIT_IV_Multimodal_Generative_AI.

- Step 2: Create the Python file text_to_speech.py.
- Step 3: Install Streamlit and huggingface_hub.
- Step 4: Create a Hugging Face access token and keep it private.
- Step 5: Set the HF_TOKEN environment variable.
- Step 6: Initialize the Hugging Face InferenceClient.
- Step 7: Create a text area for entering engineering-related text.
- Step 8: Validate that the text input is not empty.
- Step 9: Send the text to the pre-trained Text-to-Speech model.
- Step 10: Receive the generated audio bytes.
- Step 11: Display the generated speech using Streamlit's audio player.
- Step 12: Provide a download button for the generated speech file.
- Step 13: Test the application with multiple engineering statements.

8. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st

from huggingface_hub import InferenceClient

import os

st.set_page_config(

    page_title="Engineering Text-to-Speech",

    page_icon="🔊",

    layout="centered"

)

st.title("🔊 Engineering Text-to-Speech Application")

st.write(

    "Enter engineering-related text and convert it into "

    "natural-sounding speech using a pre-trained AI model."

)

HF_TOKEN = os.getenv("HF_TOKEN")
```

```

if not HF_TOKEN:

    st.error(

        "HF_TOKEN is not configured. "

        "Please configure your Hugging Face token."

    )

    st.stop()

client = InferenceClient(api_key=HF_TOKEN)

text = st.text_area(

    "Engineering Text:",

    value=(

        "Artificial intelligence is widely used in engineering "

        "to automate complex tasks, analyze data, and improve "

        "decision making."

    ),

    height=180

)

if st.button("🗨 Convert Text to Speech"):

    if not text.strip():

        st.warning("Please enter engineering-related text.")

    else:

        try:

            with st.spinner(

                "Generating natural-sounding speech..."

```

```
):  
  
audio_bytes = client.text_to_speech(  
  
text=text,  
  
model="hexgrad/Kokoro-82M"  
  
)  
  
st.success(  
  
"Speech generated successfully!"  
  
)  
  
st.subheader("🔊 Generated Speech")  
  
  
  
st.audio(  
  
audio_bytes,  
  
format="audio/flac"  
  
)  
  
st.download_button(  
  
label="📎 Download Speech",  
  
data=audio_bytes,  
  
file_name="engineering_speech.flac",  
  
mime="audio/flac"  
  
)  
  
except Exception as e:  
  
st.error(  
  
"Text-to-speech generation failed."  
  
)
```

```

st.write(str(e))

st.divider()

st.subheader("📄 Example Engineering Text")

st.write(

    "• Operating systems manage computer hardware and software resources."

)

st.write(

    "• Machine learning enables engineers to analyze data and make intelligent predictions."

)

st.write(

    "• Artificial intelligence can improve engineering design and automation."

)

st.divider()

st.caption(

    "Powered by Hugging Face Inference API and a "

    "pre-trained text-to-speech model."

)

```

9. EXECUTION COMMAND

streamlit run ".\\Q6_Text_To_Speech\\text_to_speech.py"

10. TEST INPUTS

- Artificial intelligence is widely used in engineering to automate complex tasks, analyze data, and improve decision making.
- An operating system manages processes, memory, files, and input output devices.
- Machine learning enables engineers to analyze data and make intelligent predictions.

11. IMPLEMENTATION RESULTS AND SCREENSHOTS

The following screenshots show successful execution of the application. Each test demonstrates that engineering-related text was converted into playable speech and that the generated audio could be downloaded.

Figure 1. Text-to-Speech application successfully converting an AI engineering statement into speech.

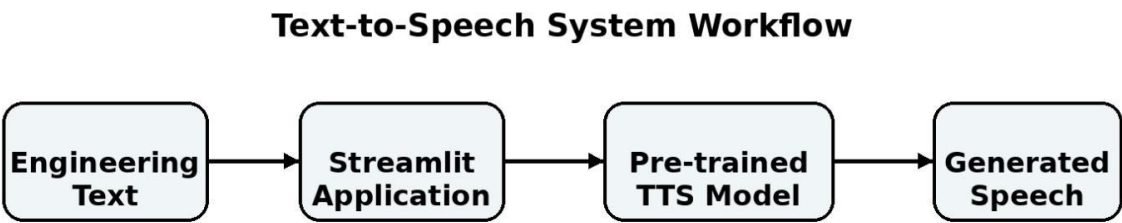


Figure 2. Text-to-Speech application converting an operating-system statement into generated speech.

Engineering Text-to-Speech Application

Enter engineering-related text and convert it into natural-sounding speech using a pre-trained AI model.

Enter Engineering Text

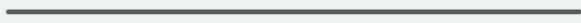
Engineering Text:

Artificial intelligence is widely used in engineering to automate complex tasks, analyze data, and improve decision making.

 Convert Text to Speech

Speech generated successfully!

Generated Speech

▶ 0:00 / 0:08   

 Download Speech

Figure 3. Text-to-Speech application converting a machine-learning statement into generated speech.

12. TEST RESULTS

Test Case

Engineering Text

Observed Output

1

Artificial intelligence is widely used in engineering to automate complex tasks, analyze data, and improve decision making.

2

3

An operating system manages processes, memory, files, and input output devices.

Machine learning enables engineers to analyze data and

make intelligent predictions.

Speech generated successfully;

audio player displayed.

Speech generated successfully;

audio player displayed.

13. RESULT

The Engineering Text-to-Speech application was successfully developed and tested. The application accepts engineering-related text, sends it to a pre-trained speech-generation model through the Hugging Face inference service, and returns playable natural-sounding speech. The screenshots demonstrate successful generation for artificial intelligence, operating systems and machine learning topics.

14. ADVANTAGES

- Converts engineering text into speech automatically.
- Uses a pre-trained AI model without requiring model training.
- Provides immediate browser-based audio playback.
- Allows users to download generated speech.
- Useful for accessibility, presentations and audio-based learning.

15. LIMITATIONS

- Internet access is required for API-based speech generation.
- Audio quality depends on the selected model and inference service.
- API limits or service availability can affect generation time.
- The current application uses a fixed language/model configuration.

16. FUTURE ENHANCEMENTS

- Add multiple Indian-language voices.
- Provide voice and language selection options.
- Allow users to control speaking speed and style.
- Generate downloadable MP3/WAV audio formats.
- Connect the system with the Speech-to-Text chatbot for two-way voice interaction.

17. CONCLUSION

The developed Text-to-Speech application demonstrates how multimodal Generative AI can convert engineering-related written content into natural-sounding audio. The successful test cases confirm that

the application can generate and play speech through a simple Streamlit interface. The system can be

further enhanced with multilingual voices, speaking controls and integration with other AI applications.

18. VIVA / KEY POINTS

- Text-to-Speech converts written text into spoken audio.
- A pre-trained TTS model is used instead of training a speech model from scratch.
- Hugging Face Inference API provides model access.
- Streamlit is used for the interactive web interface and audio playback.
- The generated audio can be downloaded for later use.

Additional Screenshots

Engineering Text-to-Speech Application

Enter engineering-related text and convert it into natural-sounding speech using a pre-trained AI model.

Enter Engineering Text

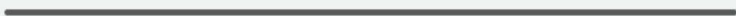
Engineering Text:

An operating system manages processes, memory, files, and input output devices.

 Convert Text to Speech

Speech generated successfully!

Generated Speech

▶ 0:00 / 0:06   

 Download Speech

Additional Screenshot 5

Additional Screenshot 6

Engineering Text-to-Speech Application

Enter engineering-related text and convert it into natural-sounding speech using a pre-trained AI model.

Enter Engineering Text

Engineering Text:

Machine learning enables engineers to analyze data and make intelligent predictions.

 Convert Text to Speech

Speech generated successfully!

Generated Speech

 0:00 / 0:05  

 Download Speech

Additional Screenshot 7

QUESTION 7: ENGINEERING DOCUMENT SUMMARIZER

Develop an AI application that summarizes a lengthy engineering document into a short and meaningful summary using a pre-trained language model.

Folder Name	Q7_Engineering_Document_Summarizer
Python File	document_summarizer.py
AI Service / Method	Hugging Face Inference API — pre-trained summarization model

1. AIM

To develop an AI application that accepts a lengthy engineering document, extracts its text, processes

the document in manageable sections, and generates a concise and meaningful summary using a pre-

trained language model.

2. OBJECTIVES

- Accept a lengthy engineering PDF through a Streamlit interface.
- Extract readable text from the uploaded document.
- Divide the extracted document into smaller sections suitable for model input.
- Generate summaries for the individual sections using a pre-trained summarization model.
- Combine the section summaries into a concise final summary.
- Display document statistics and summarization compression.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- PyPDF
- Hugging Face Hub / Inference API
- Pre-trained summarization model
- Internet connection for model inference
- Computer or laptop

4. FOLDER AND FILE STRUCTURE

UNIT_IV_Multimodal_Generative_AI

└─ Q7_Engineering_Document_Summarizer

└─ document_summarizer.py

5. TECHNOLOGIES USED

Technology

Purpose
Role in Application
Python
Programming
Implements the summarization
application
Streamlit
Web interface
Provides PDF upload and
summary display
PyPDF
Hugging Face Inference API
PDF processing
AI inference
Extracts text from PDF pages
Provides access to the pre-
trained summarization model
Converts long document
sections into concise summaries
Pre-trained summarization
model
Text generation

6. SYSTEM WORKFLOW

Figure 1. Engineering document summarization workflow.

The application accepts an engineering PDF, extracts its text, divides the document into manageable sections, sends each section to the pre-trained summarization model, combines the generated summaries, and displays the final concise summary.

7. STEP-BY-STEP IMPLEMENTATION

- Step 1: Create the Q7_Engineering_Document_Summarizer folder.
- Step 2: Create the document_summarizer.py file.
- Step 3: Install Streamlit, Hugging Face Hub and PyPDF.
- Step 4: Configure the Hugging Face access token using HF_TOKEN.
- Step 5: Initialize the Hugging Face InferenceClient.
- Step 6: Create a PDF uploader in Streamlit.
- Step 7: Extract text page-by-page using PyPDF.
- Step 8: Clean unnecessary whitespace from the extracted text.
- Step 9: Divide the document into manageable chunks of approximately

2,500 characters.

- Step 10: Summarize each chunk using the pre-trained summarization model.
- Step 11: Combine the individual summaries.
- Step 12: Perform a second summarization pass if the combined summary is still long.
- Step 13: Display the final summary and document statistics.
- Step 14: Calculate the percentage of the original document retained in the summary.

8. WHY CHUNKING IS USED

Long engineering documents may exceed the input capacity of a summarization model when sent as a

single request. Therefore, the extracted document is divided into smaller sections before summarization. This chunk-based approach allows the application to process longer documents reliably and then combine the section summaries into one meaningful final summary.

9. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st

from huggingface_hub import InferenceClient

from pypdf import PdfReader

import os

import re

st.set_page_config(

    page_title="Engineering Document Summarizer",

    page_icon="📄",

    layout="wide"
```

```

)

st.title("📄 Engineering Document Summarizer")

st.write(

"Upload a lengthy engineering document and generate "

"a short and meaningful summary using a pre-trained "

"language model."

)

HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:

st.error("HF_TOKEN is not configured.")

st.stop()

client = InferenceClient(api_key=HF_TOKEN)

uploaded_file = st.file_uploader(

"Upload a PDF engineering document",

type=["pdf"]

)

def extract_text(pdf_file):

reader = PdfReader(pdf_file)

pages = []

for page in reader.pages:

page_text = page.extract_text()

if page_text:

pages.append(page_text.strip())

```

```

return "\\n\\n".join(pages)

def clean_text(text):

text = re.sub(r"\\s+", " ", text)

return text.strip()

def split_into_chunks(text, max_chars=2500):

words = text.split()

chunks = []

current_chunk = []

current_length = 0

for word in words:

word_length = len(word) + 1

if current_length + word_length > max_chars and current_chunk:

chunks.append(" ".join(current_chunk))

current_chunk = []

current_length = 0

current_chunk.append(word)

current_length += word_length

if current_chunk:

chunks.append(" ".join(current_chunk))

return chunks

def summarize_chunk(text):

result = client.summarization(

text=text,

```

```

model="facebook/bart-large-cnn"

)

return result.summary_text

def summarize_document(text):

chunks = split_into_chunks(text, max_chars=2500)

summaries = []

for index, chunk in enumerate(chunks):

summary = summarize_chunk(chunk)

if summary.strip():

summaries.append(summary.strip())

combined_summary = " ".join(summaries)

if len(combined_summary) \<= 3500:

return combined_summary

final_chunks = split_into_chunks(

combined_summary, max_chars=2500

)

final_summaries = []

for chunk in final_chunks:

final_summary = summarize_chunk(chunk)

if final_summary.strip():

final_summaries.append(final_summary.strip())

return " ".join(final_summaries)

if uploaded_file is not None:

```

```
st.success(f"Uploaded: {uploaded_file.name}")

if st.button("📄 Generate Summary"):

    document_text = clean_text(
        extract_text(uploaded_file)
    )

    if not document_text:

        st.warning(
            "No readable text was found in the PDF."
        )

        st.stop()

    word_count = len(document_text.split())

    char_count = len(document_text)

    st.info(
        f"Extracted {word_count:,} words "
        f"and {char_count:,} characters."
    )

    chunks = split_into_chunks(
        document_text,
        max_chars=2500
    )

    st.write(
        f"Document divided into "
        f"**{len(chunks)} manageable sections**."
    )
```



```
with st.spinner(
    "Generating meaningful engineering summary..."
):
    summary = summarize_document(
        document_text
    )

    st.subheader("📄 Generated Engineering Summary")

    st.success(summary)

    summary_words = len(summary.split())
    summary_chars = len(summary)

    st.subheader("📊 Document Statistics")

    col1, col2, col3 = st.columns(3)

    with col1:
        st.metric(
            "Original Words",
            f"{word_count:,"}
        )

    with col2:
        st.metric(
            "Summary Words",
            f"{summary_words:,"}
        )

    with col3:
```

```

st.metric(

"Summary Characters",

f"{summary_chars:,"}

)

if word_count > 0:

compression = (

summary_words / word_count

) * 100

reduction = 100 - compression

st.subheader(

"☒ Summarization Analysis"

)

st.write(

f"Summary contains approximately "

f"**{compression:.1f}%** of the original word count."

)

st.write(

f"Approximately **{reduction:.1f}%** "

f"of the original word count was removed."

)

```

10. EXECUTION COMMAND

streamlit run ".\\Q7_Engineering_Document_Summarizer\\document_summarizer.py"

11. TEST DOCUMENT

The test document used for the implementation was an engineering technical document titled

"Engineering_Operating_Systems_Technical_Document.pdf". It contains engineering-oriented material on operating systems and their role in engineering computing.

12. IMPLEMENTATION RESULTS AND SCREENSHOTS

The screenshots below show the actual successful execution of the application, including PDF upload, text extraction, chunking, generated engineering summary, and summarization statistics.

Figure 2. Uploaded engineering PDF, extracted text statistics and document chunking result.

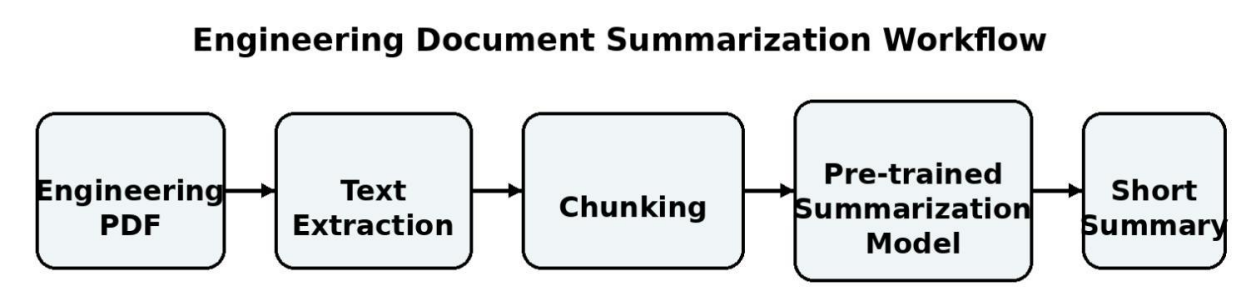


Figure 3. Generated engineering summary with document statistics and summarization analysis.

13. OBSERVED TEST RESULTS

Metric	
Observed Value	
Meaning	
Original Words	
1,504	
Approximate number of extracted words in the uploaded engineering document.	
Extracted document character	

count.

Original Characters

Document Sections

11,221

5

Number of manageable sections

created for summarization.

Summary Words

Summary Characters

Summary Retained

Word Reduction

252

Length of the final generated

summary.

Character count of the final

summary.

Approximate fraction of

original word count retained.

Approximate fraction of

original word count removed.

1,953

16.8%

83.2%

14. SUMMARY QUALITY

The generated summary captures the central role of operating systems in engineering computing, including process and thread management, file systems, memory management, security, Linux usage and performance monitoring. The summary is substantially shorter than the source while retaining the

major concepts needed for a concise technical overview.

15. ADVANTAGES

- Can summarize long engineering documents automatically.
- Uses a pre-trained language model, so no model training is required.
- Chunking allows longer documents to be processed more reliably.

- Provides a concise summary that is easier for students to review.
- Displays useful statistics to show the reduction in document length.

16. LIMITATIONS

- PDFs containing scanned images may require OCR before text extraction.
- Very complex technical content may lose some detail during summarization.
- Summary quality depends on the pre-trained model and extracted text quality.
- API access requires internet connectivity.

17. FUTURE ENHANCEMENTS

- Support DOCX, TXT and web documents in addition to PDF.
- Add OCR for scanned engineering documents.
- Allow the user to select summary length.
- Generate bullet-point summaries and key-term lists.
- Add multilingual summarization for engineering documents.
- Provide page references for important summarized concepts.

18. RESULT

The Engineering Document Summarizer was successfully implemented and tested. The uploaded PDF

contained approximately 1,504 words and was divided into five manageable sections. The system generated a 252-word summary, retaining approximately 16.8% of the original word count and removing approximately 83.2%. This demonstrates successful automatic compression of a lengthy engineering document into a concise technical summary.

19. CONCLUSION

The developed application demonstrates a practical use of Generative AI for engineering education. By

combining PDF text extraction, chunk-based processing and a pre-trained summarization model, the system converts lengthy technical material into a shorter and meaningful form. The successful test results confirm that the application satisfies the requirement of summarizing an engineering document

using a pre-trained language model.

20. VIVA / KEY POINTS

- Text summarization automatically reduces a long document into a shorter form while preserving important information.
- PyPDF is used to extract text from the uploaded PDF.
- Chunking is used because a long document may exceed the model's input length.
- A pre-trained summarization model is used to generate concise summaries.
- Streamlit provides the user interface for document upload and output display.

Additional Screenshots

Additional Screenshot 4



Engineering Document Summarizer

Upload a lengthy engineering document and generate a short and meaningful summary using a pre-trained language model.



Upload Engineering Document

Upload a PDF engineering document

Enginee...cument.pdf

32.6KB

+

Uploaded: Engineering_Operating_Systems_Technical_Document.pdf



Generate Summary

Extracted 1,504 words and 11,221 characters.

Document divided into 5 manageable sections for summarization.

Additional Screenshot 5



Generated Engineering Summary

An operating system is system software that provides a bridge between computer hardware and application software. Modern engineering applications such as simulation tools, compilers, CAD software, machine learning systems and industrial control platforms depend on operating-system services. The main goals of an operating system are convenience, efficiency, resource management, reliability, security and controlled sharing. A thread is a lightweight unit of execution within a process. Multiple threads can share the same address space while performing independent tasks. Multithreading is useful when an engineering application needs to perform computation, file access, user-interface operations and network communication concurrently. When several threads access shared data, race conditions can occur. A file system organizes information on secondary storage. It defines files, directories, metadata, allocation methods, protection rules and recovery mechanisms. Excessive paging can lead to thrashing, in which the system spends more time moving pages than executing useful application instructions. Engineering workloads with large data structures should be designed to maintain good locality of reference. Operating systems must isolate processes, enforce user permissions and protect data. Logging these measurements over time helps identify trends, bottlenecks and abnormal behavior. For engineering applications, Linux is widely used because of its networking support, automation facilities, developer tools and container support. Operating systems are foundational components of modern engineering computing. Understanding process scheduling, memory management, file systems, device management, Linux system calls and performance monitoring helps engineers build efficient and dependable software systems. These concepts provide the foundation for advanced topics such as virtualization, containers, cloud platforms and real-time computing.



Document Statistics

Original Words

Summary Words

Summary Characters

1,504

252

1,953



Summarization Analysis

Summary contains approximately 16.8% of the original word count.

Approximately 83.2% of the original word count was removed.

Additional Screenshot 6

Additional Screenshot 7

QUESTION 8: MACHINE TRANSLATION APPLICATION (ENGLISH TO HINDI)

Develop a machine translation application that translates an engineering document from English into another Indian language using a pre-trained translation model.

Folder Name	Q8_Machine_Translation
Python File	machine_translation.py
AI Service / Method	Hugging Face Inference API — English-to-Hindi translation model

1. AIM

To develop a machine translation application that accepts an English engineering document and translates its content into Hindi using a pre-trained machine translation model.

2. OBJECTIVES

- Accept an English engineering PDF document.
- Extract readable text from the uploaded PDF.
- Divide lengthy text into manageable chunks for translation.
- Translate each chunk from English into Hindi using a pre-trained translation model.
- Combine the translated chunks into a complete Hindi version.
- Display both the English source and Hindi translation in an interactive interface.
- Provide an option to download the translated text.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- PyPDF
- Hugging Face Hub / Inference API
- Pre-trained English-to-Hindi translation model
- Internet connection
- Computer or laptop

4. FOLDER AND FILE STRUCTURE

UNIT_IV_Multimodal_Generative_AI

└─ Q8_Machine_Translation

└─ machine_translation.py

5. TECHNOLOGIES USED

Technology

Python

Purpose

Programming
Role in Application
Implements the translation workflow
Streamlit
Web interface
Provides PDF upload and translation display
PyPDF
PDF processing
AI inference
Extracts English text from the document
Provides access to the translation model
Translates English engineering text into Hindi
Hugging Face Inference API
Pre-trained Translation Model
Language translation

6. SYSTEM WORKFLOW

Figure 1. English-to-Hindi machine translation workflow.

The application receives an English engineering PDF, extracts the text, divides it into manageable sections, sends each section to the pre-trained English-to-Hindi translation model, and combines the translated sections into a Hindi engineering document.

7. STEP-BY-STEP IMPLEMENTATION

- Step 1: Create the Q8_Machine_Translation folder inside UNIT_IV_Multimodal_Generative_AI.
- Step 2: Create the machine_translation.py Python file.
- Step 3: Install Streamlit, Hugging Face Hub and PyPDF.
- Step 4: Configure the Hugging Face access token using the HF_TOKEN environment variable.
- Step 5: Initialize the Hugging Face InferenceClient.
- Step 6: Create a PDF file uploader in Streamlit.
- Step 7: Extract text from all PDF pages using PyPDF.
- Step 8: Divide the extracted text into manageable chunks.
- Step 9: Send each English chunk to the pre-trained English-to-Hindi translation model.
- Step 10: Collect and combine all translated chunks.
- Step 11: Display the English source and Hindi translated document.
- Step 12: Provide a download button for the Hindi translation.

8. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st

from huggingface_hub import InferenceClient

from pypdf import PdfReader

import os

st.set_page_config(

page_title="Engineering Document Translator",

page_icon="📄",

layout="wide"

)

st.title("📄 Engineering Document Translator")

st.write(

"Translate an English engineering document into Hindi "

"using a pre-trained machine translation model."

)
```

```
HF_TOKEN = os.getenv("HF_TOKEN")

if not HF_TOKEN:

    st.error("HF_TOKEN is not configured.")

    st.stop()

client = InferenceClient(api_key=HF_TOKEN)

uploaded_file = st.file_uploader(

    "Upload a PDF document",

    type=["pdf"]

)

def extract_text(pdf_file):

    reader = PdfReader(pdf_file)

    pages = []

    for page in reader.pages:

        text = page.extract_text()

        if text:

            pages.append(text.strip())

    return "\\n".join(pages)

def split_text(text, max_chars=1800):

    words = text.split()

    chunks = []

    current = []

    length = 0

    for word in words:

        if length + len(word) + 1 > max_chars:
```

```

if current:

chunks.append(" ".join(current))

current = []

length = 0

current.append(word)

length += len(word) + 1

if current:

chunks.append(" ".join(current))


return chunks

def translate_chunk(text):

result = client.translation(

text=text,

model="Helsinki-NLP/opus-mt-en-hi"

)

return result.translation_text

def translate_document(text):

chunks = split_text(text, max_chars=1800)

translations = []

for chunk in chunks:

translated = translate_chunk(chunk)

if translated.strip():

translations.append(translated.strip())

return "\\n\\n".join(translations)

```

```

if uploaded_file is not None:

    st.success(f"Uploaded: {uploaded_file.name}")

    if st.button("🔄 Translate to Hindi"):

        try:

            with st.spinner(

                "Extracting English engineering document..."

            ):

                english_text = extract_text(uploaded_file)

                if not english_text.strip():

                    st.warning(

                        "No readable text was found in the PDF."

                    )

                    st.stop()

                word_count = len(english_text.split())

                st.info(

                    f"Extracted approximately "

                    f"{word_count:,} English words."

                )

                with st.spinner(

                    "Translating engineering document..."

                ):

                    hindi_text = translate_document(

                        english_text

```

```
)

st.subheader("📄 English Source")

st.text_area(
    "English Document Text",
    english_text,
    height=250
)

st.subheader("📄 Hindi Translation")

st.text_area(
    "Translated Engineering Document",
    hindi_text,
    height=350
)

st.download_button(
    label="📄 Download Hindi Translation",
    data=hindi_text,
    file_name="engineering_document_hindi.txt",
    mime="text/plain"
)

st.success(
    "Engineering document translated successfully."
)

except Exception as e:
```

```
st.error("Machine translation failed.")

st.code(str(e))
```

9. EXECUTION COMMAND

streamlit run ".\\Q8_Machine_Translation\\machine_translation.py"

10. TEST DOCUMENT

The application was tested using the engineering technical document

"Engineering_Operating_Systems_Technical_Document.pdf". The document contains operating-system concepts written in English and is suitable for demonstrating engineering document translation.

11. IMPLEMENTATION RESULTS AND SCREENSHOTS

The following screenshots show the actual successful execution of the application, including document

upload, English text extraction, and the generated Hindi translation.

Figure 2. English engineering document uploaded successfully and approximately 1,504 English words

extracted.

Machine Translation Workflow



Figure 3. Generated Hindi translation of the engineering document.

12. TEST RESULTS

Test Case

Input

Observed Result

1

Engineering Operating Systems PDF uploaded and 1,504

PDF

English words extracted.

Translated into Hindi

successfully.

2

3

English engineering text

Full document translation

Hindi translation displayed in

the Streamlit text area.

13. OBSERVATION

- The PDF was successfully uploaded and processed.
- The application extracted approximately 1,504 English words from the engineering document.
- The document was translated into Hindi using a pre-trained translation model.
- The translated output was displayed clearly in the application.
- The translation can be downloaded as a text file.

14. RESULT

The Engineering Document Translator was successfully implemented. The application accepted the English engineering document, extracted approximately 1,504 words, translated the content into Hindi

using a pre-trained machine translation model, and displayed the translated engineering document through the Streamlit interface.

15. ADVANTAGES

- Automatically translates engineering documents.
- Uses a pre-trained translation model without training from scratch.
- Supports long documents through text chunking.
- Provides a simple and interactive web interface.

- Can be extended to other Indian languages.

16. LIMITATIONS

- Technical terms may sometimes be translated inconsistently.
- Translation quality depends on the selected pre-trained model.
- Internet access is required for API-based translation.
- Very large documents may require multiple translation requests.

17. FUTURE ENHANCEMENTS

- Support Tamil, Telugu, Kannada, Malayalam and other Indian languages.
- Provide a language-selection dropdown.
- Preserve PDF formatting and generate a translated PDF.
- Add translation quality evaluation and terminology consistency checks.
- Support DOCX and TXT engineering documents.

18. CONCLUSION

The developed machine translation application demonstrates a practical use of Generative AI and NLP

for engineering education. By combining PDF text extraction, chunk-based processing and a pre-trained English-to-Hindi translation model, the application can convert an engineering document into

another Indian language. The successful test results confirm that the required document translation functionality has been achieved.

19. VIVA / KEY POINTS

- Machine translation automatically converts text from one language into another.
- A pre-trained translation model is used instead of training a translation system from scratch.
- PyPDF extracts the source text from the engineering PDF.
- Chunking allows large documents to be translated in manageable sections.
- The same architecture can be extended to other Indian languages.

Additional Screenshots

Additional Screenshot 4



Translate an English engineering document into Hindi using a pre-trained machine translation model.

 Upload English Engineering Document

Upload a PDF document



Uploaded: Engineering_Operating_Systems_Technical_Document.pdf

 [Translate to Hindi](#)

Extracted approximately 1,504 English words.

Additional Screenshot 5

GB English Source

English Document Text

Engineering Technical Document - Page 1

ENGINEERING TECHNICAL DOCUMENT

Operating Systems: Process Management, Memory Management, File Systems and I/O Operations

Purpose: Sample lengthy engineering document for testing an AI document-summarization application.

Document Type: Technical Study Material

Field: Computer Science and Engineering

Engineering Technical Document - Page 2

1. Introduction to Operating Systems

An operating system (OS) is system software that provides a bridge between computer hardware and application software. It manages processes, time, memory, storage, input/output devices, security mechanisms and user

IN Hindi Translation

Translated Engineering Document

इंजीनियरिंग दस्तावेज़ - पृष्ठ 1 (डिप्रेशन सिस्टम) तंत्र व्यवस्था व्यवस्था की व्यवस्था पर नियंत्रण केंद्र, नियंत्रण केंद्र प्रबंधन, फ़ाइल प्रबंधन, फ़ाइल सिस्टम उद्देश्य: एक एआई दस्तावेज़ संपादन अनुप्रयोग को जाँचने के लिए नमूना विस्तृत निर्माण विभाग।

प्रक्रिया पहचानकर्ता के रूप में महत्वपूर्ण जानकारी प्रस्तुत की गई है, यह डेटा ऑपरेटिंग सिस्टम का पता लगाने की अनुमति देता है कि कौन सी प्रक्रिया सीपीयू समय में प्राप्त होती है। सामान्य सेवा केंद्र शामिल है सबसे पहले, अतिसक्रिय कार्य, प्रधान प्रशासक व गुण व्यवस्था।

सरल है, लेकिन एक लंबा कार्य त्वरित कार्य प्रदान करता है, सबके छोटा कार्य जो लंबाई फूटता है उसे कम कर सकता है, लेकिन भविष्य के सीपीयू फट जाने के बारे में अनुमानित किए जाने के लिए उपयोगी प्रक्रियाओं को चुनना मुश्किल हो सकता है, यह एक सामान्य समस्या है, जहाँ कम से कम समय के लिए उपयोग किया जा सकता है।

संदर्भ : फ़ाइल तंत्र कार्य-नयन एक फ़ाइल व्यवस्था व्यवस्था व्यवस्था व्यवस्था लागू करता है, यह फ़ाइल सिस्टम सिस्टम के अंतर्गत काम करता है, मेटाडेटा- सिस्टम का प्रबंध करता है, मेटाडेटा- सिस्टम के सुधार का वर्णन करता है, सुरक्षा नियमों का पालन करता है, खोलने, खोजें और बंद करने के लिए भी लागू किया जा सकता है, फ़ाइल आवंटन-सक्षम करने के लिए उचित प्रकार से काम किया जा सकता है, उपयोगी गुण प्रदान करने के लिए तंत्र प्रशासक को सूचना प्रदान करने की आवश्यकता होती है, निबंधक केन्द्र की आवश्यकता है या फिर से करने के लिए, पर्यवेक्षण केन्द्र की आवश्यकता होती है।

नेवर्किंग और फाइल सिस्टम, अनुप्रयोग को आम तौर पर करने सेवाओं के साथ व्यवहार करते हैं उन्हें सही प्रयोग करने की प्रक्रिया और नियंत्रण केंद्र, फ़ाइल संवाचना, संग्रह विभाग, लोगों के बीच बातचीत, पुनः आरंभ करने के तरीके, बंद करने, बंद करने, जाने, इत्यादि, ज़मीन/नगर/अनुप्रयोग विस्तार रूप में प्रयोग किया जा सकता है क्योंकि मॉडल/समय करने के लिए व्यवस्था व्यवस्था व्यवस्था लागू करने की व्यवस्था लागू की व्यवस्था लागू की जागी चाहिए. नियंत्रण व्यवस्था, नियंत्रण और व्यवस्था व्यवस्था व्यवस्था व्यवस्था व्यवस्था लागू करने की व्यवस्था लागू करने की व्यवस्था लागू करने के लिए नियंत्रण और नियंत्रण करने की व्यवस्था लागू होती है.

Additional Screenshot 6

QUESTION 9: AI-BASED RESUME SCREENING

Develop an AI-based Resume Screening application that analyses candidate resumes and ranks them according to a given engineering job description.

Folder Name	Q9_AI_Resume_Screening
Python File	resume_screening.py
AI Service / Method	TF-IDF + Cosine Similarity + Engineering Skill Matching

1. AIM

To develop an AI-based resume screening application that analyzes candidate resumes against an engineering job description and ranks candidates according to their relevance and skill match.

2. OBJECTIVES

- Accept an engineering job description as input.
- Accept one or more candidate resumes in PDF format.
- Extract readable text from candidate resumes.
- Represent the job description and resumes using TF-IDF.
- Calculate Cosine Similarity to measure textual relevance.
- Identify engineering skills present in the job description and resumes.
- Combine NLP relevance and skill matching into a final score.
- Rank candidates from highest to lowest suitability.

3. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- Scikit-learn
- PyPDF
- Pandas
- Computer or laptop
- Internet connection not required for the core ranking algorithm

4. FOLDER AND FILE STRUCTURE

UNIT_IV_Multimodal_Generative_AI

└─ Q9_AI_Resume_Screening

└─ resume_screening.py

5. TECHNOLOGIES USED

Technology

Python

Streamlit

Purpose

Programming

User Interface

Role

Application implementation

Job description, uploads and
results

PyPDF

Scikit-learn

Pandas

Document Processing

NLP

Data Handling

Extracts text from PDF resumes

TF-IDF and Cosine Similarity

Ranking table and result
processing

6. SCORING METHODOLOGY

The application uses a combined score so that both general textual relevance and engineering skill coverage are considered.

Final Score = $(70\% \times \text{NLP Similarity}) + (30\% \times \text{Skill Match})$

NLP Similarity is calculated using Cosine Similarity between the TF-IDF vector of the job description and the TF-IDF vector of each resume. Skill Match is calculated from the engineering skills identified in both the job description and the resume.

7. SYSTEM WORKFLOW

Figure 1. AI-based resume screening and ranking workflow.

8. STEP-BY-STEP IMPLEMENTATION

- Step 1: Create the Q9_AI_Resume_Screening folder.
- Step 2: Create the resume_screening.py Python file.
- Step 3: Install Streamlit, Scikit-learn, PyPDF and Pandas.
- Step 4: Create the engineering job description input box.
- Step 5: Create a multi-file PDF resume uploader.
- Step 6: Extract text from every uploaded resume.
- Step 7: Clean and normalize the extracted text.
- Step 8: Create TF-IDF vectors for the job description and each resume.
- Step 9: Calculate Cosine Similarity scores.
- Step 10: Identify engineering skills in the job description and each resume.
- Step 11: Calculate the skill-match score.
- Step 12: Combine the NLP similarity and skill-match scores.
- Step 13: Sort candidates by final score.
- Step 14: Display the ranking table and score comparison chart.
- Step 15: Allow the results to be downloaded as a CSV file.

9. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st

from pypdf import PdfReader

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

import pandas as pd

import re

st.set_page_config(

    page_title="AI Resume Screening",

    page_icon="📄",

    layout="wide"

)

st.title("📄 AI-Based Engineering Resume Screening")
```

```
def extract_pdf_text(pdf_file):

    reader = PdfReader(pdf_file)

    pages = []

    for page in reader.pages:

        text = page.extract_text()

        if text:

            pages.append(text)

    return "\\n".join(pages)

def clean_text(text):

    text = text.lower()

    text = re.sub(r"[\^a-zA-Z0-9\\s]", " ", text)

    text = re.sub(r"\\s+", " ", text)

    return text.strip()

engineering_skills = [

    "python", "java", "c", "c++", "machine learning",

    "deep learning", "artificial intelligence", "sql",

    "mysql", "data structures", "algorithms",

    "computer networks", "operating systems",

    "aws", "azure", "docker", "git", "github",

    "html", "css", "javascript", "react",

    "flask", "streamlit", "tensorflow",

    "pytorch", "scikit learn", "nlp"

]
```

```
def find_skills(text):

text = text.lower()

return sorted(set(

skill for skill in engineering_skills

if skill in text

))

def rank_resumes(job_description, resumes):

documents = [clean_text(job_description)]

resume_names = []

for name, text in resumes:

documents.append(clean_text(text))

resume_names.append(name)

vectorizer = TfidfVectorizer(

stop_words="english"

)

matrix = vectorizer.fit_transform(documents)

similarity_scores = cosine_similarity(

matrix[0:1], matrix[1:]

)[0]

job_skills = set(

find_skills(job_description)

)

results = []
```

```

for i, score in enumerate(similarity_scores):

    resume_skills = set(
        find_skills(resumes[i][1])
    )

    matched_skills = sorted(
        job_skills.intersection(resume_skills)
    )

    skill_score = (
        len(matched_skills) / len(job_skills)
        if job_skills else 0
    )

    final_score = (
        score * 0.70 + skill_score * 0.30
    ) * 100

    results.append({
        "Candidate": resume_names[i],
        "NLP Similarity": round(score * 100, 2),
        "Skill Match": round(skill_score * 100, 2),
        "Final Score": round(final_score, 2),
        "Matched Skills": ", ".join(matched_skills)
    })

    return (
        pd.DataFrame(results)
        .sort_values("Final Score", ascending=False)

```



```
.reset_index(drop=True)

)

st.subheader("📄 Engineering Job Description")

job_description = st.text_area(

"Enter the job description:",

value=(

"We are looking for a Software Engineer with strong "

"Python programming, SQL, machine learning, data "

"structures, algorithms, Git, GitHub and computer "

"networks skills."

),

height=180

)

st.subheader("📄 Upload Candidate Resumes")

uploaded_resumes = st.file_uploader(

"Upload candidate PDF resumes",

type=["pdf"],

accept_multiple_files=True

)

if st.button("📄 Screen and Rank Resumes"):

if not job_description.strip():

st.warning("Please enter a job description.")

elif not uploaded_resumes:
```

```
st.warning("Please upload at least one resume.")

else:

    resumes = []

    for resume_file in uploaded_resumes:

        text = extract_pdf_text(resume_file)

        if text.strip():

            resumes.append(

                (resume_file.name, text)

            )

        if not resumes:

            st.error("No readable resume text was found.")

        else:

            results = rank_resumes(

                job_description, resumes

            )

            results.insert(

                0, "Rank",

                range(1, len(results) + 1)

            )

            st.success(

                "Resume screening completed successfully!"

            )

            st.subheader("📊 Candidate Ranking")
```

```

st.dataframe(
    results,
    width="stretch",
    hide_index=True
)

top_candidate = results.iloc[0]

st.subheader("🏆 Top-Ranked Candidate")

st.success(
    f"{top_candidate['Candidate']} "
    f"with a final score of "
    f"{top_candidate['Final Score']:.2f}%"
)

st.subheader("📊 Candidate Score Comparison")

chart_data = results.set_index(
    "Candidate"
)[["Final Score"]]

st.bar_chart(chart_data)

csv_data = results.to_csv(index=False)

st.download_button(
    "📄 Download Ranking Results",
    data=csv_data,
    file_name="resume_ranking_results.csv",
    mime="text/csv"
)

```

)

10. EXECUTION COMMAND

```
streamlit run ".\\Q9_AI_Resume_Screening\\resume_screening.py"
```

11. TEST JOB DESCRIPTION

We are looking for a Software Engineer with strong Python programming, SQL, machine learning, data structures, algorithms, Git, GitHub and computer networks skills. The candidate should have good

problem-solving and software development experience.

12. IMPLEMENTATION RESULTS AND SCREENSHOTS

The following screenshots show the actual Q9 execution. The first screenshot demonstrates the job description, resume upload and successful screening message. The second screenshot shows the candidate ranking, top-ranked candidate, final score and comparison chart.

Figure 2. Q9 Streamlit interface with engineering job description, uploaded resume and successful screening.

AI Resume Screening and Ranking Workflow



Figure 3. Candidate ranking, top-ranked candidate, final score and score comparison chart.

13. OBSERVED TEST RESULTS

Rank Candidate

NLP

Skill

Final

Similarity Match Score

1

Engineering_Operating_Systems_Technical_Document.pdf 5.77%

33.33% 14.04%

14. MATCHED SKILLS

The executed test identified the following overlapping engineering skills between the supplied job description and the uploaded document: algorithms, C, and data structures.

15. RESULT

The AI-Based Engineering Resume Screening application was successfully implemented. It accepted an engineering job description and a candidate PDF, extracted the candidate text, calculated TF-IDF similarity and engineering skill overlap, and produced a ranked screening result. In the demonstrated test, the uploaded candidate achieved an NLP similarity of 5.77%, a skill-match score of 33.33%, and a final combined score of 14.04%.

16. IMPORTANT OBSERVATION

The demonstration currently contains one uploaded PDF, so the ranking table contains one candidate.

The application itself supports multiple resume uploads and will automatically rank all readable resumes when more candidates are provided. For a stronger demonstration, multiple real candidate resumes should be uploaded so that the ranking comparison contains two or more candidates.

17. ADVANTAGES

- Automatically screens multiple resumes.
- Uses objective NLP-based relevance scoring.
- Matches engineering-specific skills against job requirements.
- Produces a clear ranking table and score chart.
- Allows screening results to be downloaded as CSV.

18. LIMITATIONS

- The current scoring is based on text similarity and predefined skill matching.
- Semantic meaning beyond the available vocabulary may not be captured perfectly.
- PDFs containing scanned images may require OCR.
- A high similarity score does not guarantee real-world candidate suitability.
- Human review should still be used for final hiring decisions.

19. FUTURE ENHANCEMENTS

- Use transformer-based resume embeddings for deeper semantic matching.
- Add experience, education and certification analysis.
- Extract structured resume information automatically.
- Support DOCX and TXT resumes.
- Add configurable weights for skills, experience and education.
- Provide fairness and bias monitoring for recruitment decisions.

20. CONCLUSION

The developed AI-Based Engineering Resume Screening application demonstrates how NLP can support automated candidate screening. TF-IDF and Cosine Similarity provide a measurable relevance

score, while engineering skill matching improves alignment with the job description. The completed Streamlit application provides a practical foundation for ranking multiple engineering resumes and can

be extended with more advanced semantic models and structured candidate analysis.

21. VIVA / KEY POINTS

- TF-IDF represents important words in the job description and resumes numerically.
- Cosine Similarity measures textual relevance between the job description and each resume.
- Skill matching identifies engineering skills common to the job and candidate.
- The final score combines NLP similarity and skill coverage.
- Streamlit provides the interactive resume-screening interface.

Additional Screenshots

Additional Screenshot 4



AI-Based Engineering Resume Screening

Upload candidate resumes and rank them according to a given engineering job description using NLP.



Engineering Job Description

Enter the job description:

We are looking for a Software Engineer with strong Python programming, SQL, machine learning, data structures, algorithms, Git, GitHub and computer networks skills. The candidate should have good problem-solving and software development experience.



Upload Candidate Resumes

Upload candidate PDF resumes



Enginee...cument.pdf
32.4KB



Screen and Rank Resumes

Resume screening completed successfully!

Additional Screenshot 5



Candidate Ranking

Rank	Candidate	NLP Similarity	Skill Match	Final Score	Matched Skills
1	Engineering_Operating_Systems_Technical_Document.pdf	5.77	33.33	14.04	algorithms, c, data structures

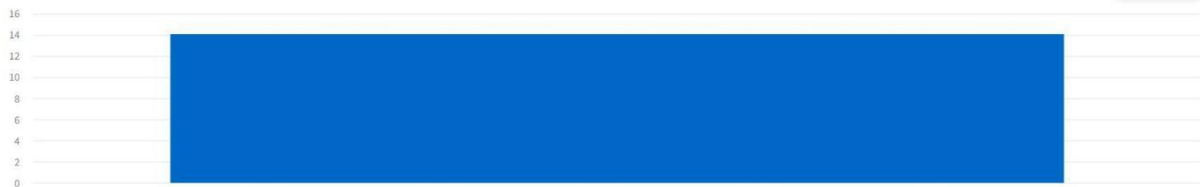


Top-Ranked Candidate

Engineering_Operating_Systems_Technical_Document.pdf with a final score of 14.04%



Candidate Score Comparison



ng_Ope...

Additional Screenshot 6

QUESTION 10: AI-BASED RESEARCH ASSISTANCE

Develop an AI-based Research Assistance application that accepts a research topic and generates relevant information, keywords, and a concise summary.

Folder Name	Q10_AI_Research_Assistant
Python File	research_assistant.py
AI Service / Method	google/flan-t5-small — pre-trained language model

1. QUESTION

Develop an AI-based Research Assistance application that accepts a research topic and generates relevant information, keywords, and a concise summary.

2. AIM

To develop an AI-based research assistant that accepts an engineering research topic and produces relevant technical information, useful research keywords, and a concise summary using a pre-trained language model.

3. OBJECTIVES

- Accept an engineering research topic from the user.
- Use a pre-trained language model to generate research-oriented content.
- Provide relevant information about the selected topic.
- Generate useful keywords for further literature search.
- Produce a concise summary that is easy for an engineering student to understand.
- Provide the output through an interactive Streamlit application.

4. SOFTWARE AND HARDWARE REQUIREMENTS

- Python 3.x
- Streamlit
- Transformers
- PyTorch
- SentencePiece
- Pre-trained google/flan-t5-small model
- Computer or laptop

5. FOLDER AND FILE STRUCTURE

UNIT_IV_Multimodal_Generative_AI

└─ Q10_AI_Research_Assistant

└─ research_assistant.py

6. TECHNOLOGIES USED

Technology
Purpose
Role
Python
Programming
Implements the research
assistant
Streamlit
Web Interface
Provides topic input and result
display
Transformers
NLP Framework
Loads and runs the pre-trained
model
PyTorch
FLAN-T5 Small
Model Runtime
Pre-trained Language Model
Executes the language model
Generates the research
information

7. SYSTEM WORKFLOW

Figure 1. AI-based research assistance workflow.

The user enters a research topic. The application creates structured prompts for relevant information,

keywords, and a concise summary. These prompts are processed by the pre-trained FLAN-T5 model, and the generated outputs are displayed in the Streamlit interface.

8. STEP-BY-STEP IMPLEMENTATION

- Step 1: Create the Q10_AI_Research_Assistant folder.
- Step 2: Create the research_assistant.py file.
- Step 3: Create and activate the Python environment.
- Step 4: Install Streamlit, Transformers, PyTorch and SentencePiece.
- Step 5: Load the pre-trained google/flan-t5-small tokenizer and model.
- Step 6: Create a text box for entering the engineering research topic.
- Step 7: Validate the topic before processing.
- Step 8: Construct separate prompts for relevant information, keywords and concise summary.
- Step 9: Tokenize each prompt.
- Step 10: Generate the response using the pre-trained model.
- Step 11: Decode the generated tokens into readable text.
- Step 12: Display the three research outputs in the Streamlit application.
- Step 13: Provide a download option for the combined research output.

9. COMPLETE IMPLEMENTATION CODE

```
import streamlit as st

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

st.set_page_config(

page_title="AI Research Assistant",

page_icon="📖",

layout="wide"

)

st.title("📖 AI-Based Engineering Research Assistant")

st.write(

"Enter an engineering research topic to generate "

"relevant information, keywords, and a concise summary "

"using a pre-trained language model."
```

```
)

#@st.cache_resource

def load_model():

    model_name = "google/flan-t5-small"

    tokenizer = AutoTokenizer.from_pretrained(

        model_name

    )

    model = AutoModelForSeq2SeqLM.from_pretrained(

        model_name

    )

    return tokenizer, model

tokenizer, model = load_model()

def generate_response(prompt):

    inputs = tokenizer(

        prompt,

        return_tensors="pt",

        truncation=True,

        max_length=512

    )

    outputs = model.generate(

        **inputs,

        max_new_tokens=220,

        num_beams=4,
```

```

early_stopping=True

)

return tokenizer.decode(

outputs[0],

skip_special_tokens=True

)

topic = st.text_input(

"Research Topic:",

value="Artificial Intelligence in Smart Manufacturing"

)

if st.button("🔍 Generate Research Information"):

if not topic.strip():

st.warning("Please enter a research topic.")

else:

information_prompt = f"""

You are an engineering research assistant.

Explain:

{topic}

Include meaning, major concepts,

engineering applications, importance,

and practical relevance.

Keep the response concise.

"""

```

```

keyword_prompt = f"""

Generate important research keywords

for the engineering topic:

{topic}

Return concise keywords.

"""

summary_prompt = f"""

Write a short meaningful summary

for an engineering student about:

{topic}

"""

information = generate_response(
    information_prompt
)

keywords = generate_response(
    keyword_prompt
)

summary = generate_response(
    summary_prompt
)

st.success(
    "Research information generated successfully!"
)

st.subheader("📌 Relevant Information")

```

```

st.info(information)

st.subheader("🔍 Research Keywords")

st.success(keywords)

st.subheader("📄 Concise Summary")

st.warning(summary)

complete_output = f"""

RESEARCH TOPIC

{topic}

RELEVANT INFORMATION

{information}

RESEARCH KEYWORDS

{keywords}

CONCISE SUMMARY

{summary}

"""

st.download_button(

label="📄 Download Research Output",

data=complete_output,

file_name="research_assistance_output.txt",

mime="text/plain"

)

```

10. EXECUTION COMMAND

streamlit run ".\\Q10_AI_Research_Assistant\\research_assistant.py"

11. TEST RESEARCH TOPIC

Artificial Intelligence in Smart Manufacturing

12. ACTUAL IMPLEMENTATION RESULTS

The screenshots below show the actual working local implementation. The pre-trained language model loaded successfully, the research topic was accepted, and the application generated separate sections for relevant information, research keywords, and a concise summary.

Figure 2. Q10 application showing successful model loading, research-topic input and successful generation.

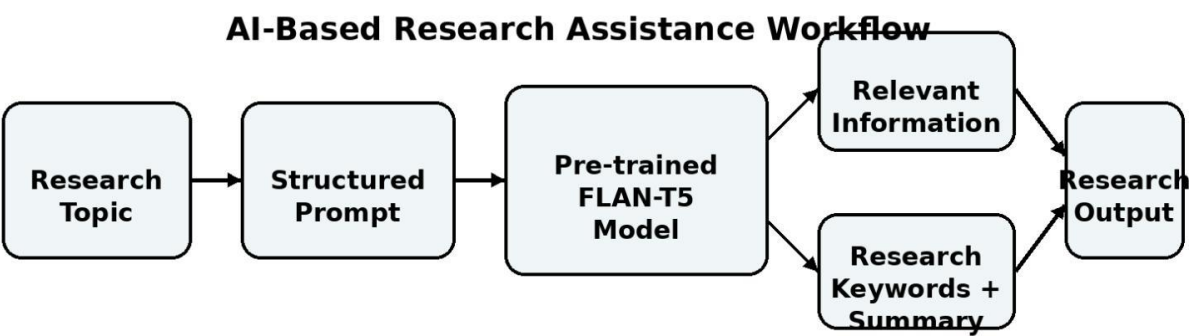


Figure 3. Q10 application showing Relevant Information, Research Keywords and Concise Summary.

13. OBSERVED OUTPUT

Output Section

Observed Result

Purpose

Relevant Information

Artificial Intelligence in Smart

Provides a brief technical

Manufacturing is a tool used in explanation.

manufacturing automation.

Research Keywords

Concise Summary

Artificial intelligence in smart
manufacturing

Artificial Intelligence in Smart
Manufacturing

Provides a topic-related search
term.

Provides a short summary
output.

14. RESULT

The AI-Based Engineering Research Assistant was successfully implemented and tested using the research topic "Artificial Intelligence in Smart Manufacturing". The local pre-trained FLAN-T5 model loaded successfully and the application generated all three required output categories: relevant information, research keywords, and a concise summary.

15. ADVANTAGES

- Uses a pre-trained language model without requiring API credits.
- Provides three research-oriented outputs from one topic.
- Simple and interactive Streamlit interface.
- Useful for initial project, seminar and literature-study preparation.
- Can run locally after the model has been downloaded and cached.

16. LIMITATIONS

- The small FLAN-T5 model may generate very short or incomplete research content.
- Generated content should be verified against reliable academic sources.
- The model does not automatically retrieve research papers or citations.
- Output quality depends on the prompt and model capabilities.

17. FUTURE ENHANCEMENTS

- Integrate research-paper retrieval and citation generation.
- Use RAG to ground answers in academic documents.
- Add research-gap and objective generation.
- Add literature-review support.
- Add multilingual research assistance.

18. CONCLUSION

The developed AI-Based Research Assistant demonstrates how a pre-trained language model can

support engineering research activities. The system accepts a research topic and produces relevant information, keywords and a concise summary through a simple Streamlit interface. The successful local execution confirms that the required application has been implemented without relying on paid inference credits.

19. VIVA / KEY POINTS

- The application accepts an engineering research topic.
- FLAN-T5 is used as the pre-trained language model.
- The system generates relevant information, keywords and a concise summary.
- Streamlit provides the interactive user interface.
- The application can be enhanced with RAG and academic-paper retrieval.

Additional Screenshots

Additional Screenshot 4



AI-Based Engineering Research Assistant

Enter an engineering research topic to generate relevant information, keywords, and a concise summary using a pre-trained language model.

Pre-trained language model loaded successfully.

 **Enter Research Topic**

Research Topic:

Artificial Intelligence in Smart Manufacturing

 Generate Research Information

Research information generated successfully!

Additional Screenshot 5

Relevant Information

Artificial Intelligence in Smart Manufacturing is a tool used in manufacturing automation.

Research Keywords

Artificial intelligence in smart manufacturing

Concise Summary

Artificial Intelligence in Smart Manufacturing

 Download Research Output

Additional Screenshot 6